



Dividendes : Analyse, Stratégies et Modélisation

Mémoire de recherche - Stage de fin d'études

Cursus : Ingénieur mathématiques appliquées modélisation mathématique
pour la finance

Étudiant : James du Peloux

Septembre 2025 – Mars 2026

Table des matières

Remerciements	3
1 Courbe de bêta dividendes et volatilité de dividendes	4
1.1 Objectif et Périmètre	4
1.2 Méthodologie des régressions linéaires (bêta par maturité)	4
1.2.1 Spécifications	4
1.2.2 Résultats	5
1.2.3 Contrôle par données de marché	6
1.3 Volatilité de dividendes : régressions et calibration SABR	7
1.3.1 Spécifications	7
1.3.2 Résultats (Euro Stoxx 50, mensuel)	7
1.3.3 Lecture, limites et décisions	8
1.3.4 Calibration SABR (description mathématique)	8
2 Stratégies sur dividendes : <i>mean-reversion</i> du bêta et <i>steepener</i> de pente	12
2.1 Motivation : pourquoi trader les dividendes ?	12
2.2 Stratégie n°1 : <i>mean-reversion</i> du bêta DivSwap vs Spot	13
2.2.1 Idée et pipeline	13
2.2.2 Optimisation du <i>pipeline</i> (score pénalisé, grille et recherche aléatoire)	14
2.2.3 Validation par apprentissage automatique (train, validation, test)	16
2.2.4 Pourquoi la stratégie ne fonctionne pas.	17
2.3 Stratégie n°2 : <i>steepener</i> de la pente de bêta (maturité constante)	19
2.3.1 Idée et implémentation	19
2.3.2 Pourquoi la stratégie ne fonctionne pas.	19
3 Modèle de structure par terme des dividendes (DTS)	21
3.1 Vue d'ensemble et idée directrice	21
3.2 Étape 1 — Maturités constantes et Actualisation	22
3.3 Étape 2 — Mesures par maturité (log-ratios)	23
3.4 Étape 3 — 2 facteurs latents et filtre de Kalman	24
3.4.1 Diagnostics et sorties	27

3.5	Étape 4 — Diagnostics et visualisations	29
3.6	Étape 5 — Ratio prix/dividende implicite et nowcasting	30
	Bibliographie	32
	Annexe	34

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude à mon maître de stage et tuteur, *Responsable du contrôle de la qualité du marquage FO et de l'IPV*. Je le remercie sincèrement pour la confiance qu'il m'a témoignée dès mon arrivée, pour sa patience et pour ses précieux conseils. Grâce à son accompagnement unique, j'ai pu non seulement développer les compétences techniques nécessaires à ma carrière, mais aussi affiner et consolider mon projet professionnel. Son soutien, ses recommandations et ses relectures attentives m'ont également été d'une aide précieuse dans l'élaboration de ce rapport.

Mes remerciements s'adressent ensuite à chacun des professionnels du *Valuation Group*, mais aussi plus largement ceux de la salle des marchés, que j'ai eu la chance de côtoyer, qui m'ont accueilli chaleureusement et m'ont accordé leur temps. Cette immersion totale m'a permis de me confronter aux réalités du métier et de transformer mes connaissances théoriques en une expérience concrète et enrichissante. Ce stage a été pour moi une véritable révélation professionnelle.

Je souhaite également remercier toutes les personnes rencontrées au cours de cette aventure, et plus particulièrement les autres stagiaires et alternants. Nos échanges, conseils et soutien mutuel ont rendu ce quotidien agréable et stimulant et m'ont permis d'évoluer dans un environnement de travail bienveillant.

Enfin, je tiens à remercier mes professeurs et l'équipe pédagogique de CY Tech pour la qualité de leur enseignement tout au long de ce cursus, qui ont su me transmettre les outils et le savoir-faire indispensables à une insertion professionnelle réussie, me permettant ainsi d'aborder ce stage de fin d'études avec sérénité et ambition.

Introduction

Les dividendes occupent une place singulière dans l'univers des dérivés actions. Longtemps traités comme un simple paramètre d'entrée des modèles de valorisation, une hypothèse que l'on fixe pour pouvoir pricer autre chose, ils constituent aujourd'hui une classe d'exposition à part entière, négociée sous forme de *Dividend Swaps* et de *Dividend Futures* sur les principaux indices mondiaux. Ce double statut fait tout l'intérêt de leur étude : le dividende anticipé est à la fois une donnée cotée par maturité et une hypothèse de marquage dont dépend la valorisation d'une large part des produits structurés.

Ce mémoire clôt un stage de fin d'études de six mois effectué au sein du *Valuation Group*, en salle des marchés, sous la responsabilité du Responsable du contrôle de la qualité du marquage *Front Office* et de l'*Independent Price Verification*. La mission du service consiste à vérifier, indépendamment des équipes de *trading*, que les paramètres de valorisation reflètent les conditions de marché, conformément aux exigences d'IFRS 13 et de l'article 105 du règlement CRR. Les dividendes y constituent un paramètre critique et difficile à contrôler : cotations hétérogènes selon les maturités, liquidité concentrée sur les échéances courtes, et une part significative de la courbe qui relève du marquage plutôt que de l'observation directe.

De ce constat découle la question qui structure ce travail : **comment objectiver la structure par terme des dividendes d'un indice, de manière suffisamment robuste pour servir de référence indépendante aux marquages du *Front Office* ?** Trois angles complémentaires, qui suivent aussi la chronologie du stage, y répondent. Le premier chapitre mesure empiriquement, par régressions linéaires, la sensibilité des *DivSwaps* au *Spot* maturité par maturité, puis transpose la démarche aux volatilités avec une calibration SABR du *smile*. Le deuxième cherche à monétiser ces mesures à travers deux stratégies quantitatives ; aucune ne résiste à une validation hors échantillon, et l'analyse de ces échecs oriente la suite. Le troisième propose un cadre structurel : un modèle de structure par terme des dividendes estimé par filtre de Kalman sur deux facteurs latents, validé par un exercice de *nowcasting* de l'indice.

Le choix de l'Euro Stoxx 50 comme périmètre principal tient à la profondeur de son marché de dérivés de dividendes, les autres indices servant de points de comparaison. L'ensemble a été développé en Python sur une dizaine d'années de données quotidiennes de dérivés de dividendes, complétées par les courbes de taux de la Banque centrale européenne. Les codes et les journaux détaillés sont reportés en annexes.

Chapitre 1

Courbe de bêta dividendes et volatilité de dividendes

1.1 Objectif et Périmètre

L'objectif est (i) d'estimer, par régressions linéaires, une courbe de bêta reliant les rendements de Dividends Swaps (DivSwaps) aux rendements du Spot pour un ensemble d'échéances (1Y ... 8Y), et (ii) d'analyser la volatilité de dividendes via la relation entre $\Delta\sigma_{\text{div}}$ et $\Delta\sigma_{\text{spot}}$, en comparant une volatilité ATM¹ implicite des Call et Put et une volatilité ATM issue d'une calibration SABR sur options de DivSwaps.²

1.2 Méthodologie des régressions linéaires (bêta par maturité)

1.2.1 Spécifications

Pour une maturité T , on estime sur fenêtre temporelle $\mathcal{T}$:

$$r_{T,t}^{\text{Div}} = \alpha_T + \beta_T r_t^{\text{Spot}} + \varepsilon_{T,t}, \quad t \in \mathcal{T}, \quad (1.1)$$

où $r_{T,t}^{\text{Div}}$ est le rendement en % du DivSwap d'échéance T , et r_t^{Spot} le rendement en % du Spot (indice sous-jacent). Les régressions sont conduites par OLS³ avec erreurs-types robustes (HC1), par *bucket* de maturité, et à deux fréquences de pas de temps : mensuelle et hebdomadaire. Les estimations sont confrontées à des niveaux de marché pour contrôle de cohérence.

1. *At-The-Money*

2. Les graphiques, tables complètes de régression et diagnostics détaillés sont reportés en annexes.

3. *Ordinary Least Squares*

La régression linéaire par la méthode des moindres carrés ordinaires (*Ordinary Least Squares*, OLS) consiste à estimer le paramètre $\beta \in \mathbb{R}^k$ du modèle

$$y = X\beta + \varepsilon,$$

où $y \in \mathbb{R}^n$ est le vecteur des observations, $X \in \mathbb{R}^{n \times k}$ la matrice de régressions et ε un vecteur d'erreurs de moyenne conditionnelle nulle. L'estimateur OLS s'écrit

$$\hat{\beta}_{\text{OLS}} = (X^\top X)^{-1} X^\top y.$$

Lorsque l'hypothèse d'homoscédasticité des erreurs est violée, les erreurs-types classiques deviennent inconsistantes. Les erreurs-types robustes de type HC1 corrigent ce biais en utilisant une matrice de variance-covariance fondée sur les résidus estimés, donnée par

$$\widehat{\text{Var}}(\hat{\beta})_{\text{HC1}} = \frac{n}{n-k} (X^\top X)^{-1} \left(\sum_{i=1}^n \hat{\varepsilon}_i^2 x_i x_i^\top \right) (X^\top X)^{-1},$$

permettant une inférence valide en présence d'hétéroscédasticité de forme inconnue.

1.2.2 Résultats

Sur le **SX5E**⁴, en mensuel, la corrélation DivSwap/Spot croît avec la maturité (env. 0.43 à 1Y jusqu'à ≈ 0.86 à 7Y), tandis que le bêta se rapproche rapidement de 1 dès 2Y ($\beta_{2Y} \approx 1.16$, $\beta_{3Y} \approx 1.14$, puis $\beta_{5Y} \approx 1.08$, $\beta_{7Y} \approx 1.02$).

En hebdomadaire sur le SX5E, la corrélation progresse également (env. 0.24 à 1Y jusqu'à 0.90 à 7Y, puis léger tassement à 8Y), avec un β_T qui converge vers 1 entre 3Y et 7Y ($\beta_{3Y} \approx 0.96$, $\beta_{4Y} \approx 1.03$, $\beta_{5Y} \approx 1.06$, $\beta_{7Y} \approx 1.04$, $\beta_{8Y} \approx 0.90$).

Sur le **SPX**⁵, la corrélation est modérée (env. $0.47 \rightarrow 0.75$ de 1Y à 7Y), et les β_T sont plus faibles que sur le SX5E ($\beta_{1Y} \approx 0.26$, $\beta_{3Y} \approx 0.70$, $\beta_{5Y} \approx 0.64$, $\beta_{7Y} \approx 0.61$).

Sur le **HSCEI**⁶ et le **NKY**⁷, les corrélations sont plus basses au court terme (p. ex. 0.15 à 1Y pour le HSCEI; 0.08 à 1Y pour le NKY) et augmentent avec la maturité (jusqu'à 0.68–0.82), mais les β_T restent significativement inférieurs à 1 aux courts et moyens termes (p. ex. $\beta_{3Y} \approx 0.41$ HSCEI; $\beta_{4Y} \approx 0.64$ NKY).

Lecture. (i) **Courbe croissante** de corrélation et de β_T avec la maturité sur le SX5E $\Rightarrow$ la composante *Yield* domine progressivement la composante *Cash* dans l'allocation dividendes (intuitivement, les horizons plus longs lissent l'aléa de dividendes). (ii) $\beta_T \approx 1$ au milieu/long terme sur le SX5E : mouvements de DivSwaps et du Spot co-intègrent

4. Euro Stoxx 50

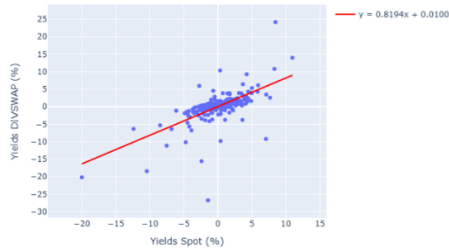
5. S&P 500

6. Hang Seng China Entreprises Index

7. Nikkei 225

Term	1Y	2Y	3Y	4Y	5Y	6Y	7Y	8Y
Correl	0.24	0.64	0.81	0.88	0.89	0.9	0.9	0.86
Coeff	0.27	0.82	0.96	1.03	1.06	1.05	1.04	0.9

Yield correlation between S&P 500 and Client Divswap (2Y) : $r = 0.64$



Yield correlation between S&P 500 and Client Divswap (1Y) : $r = 0.24$

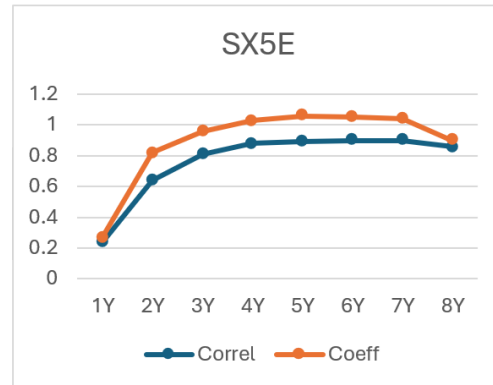
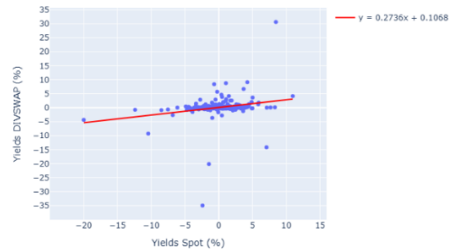


FIGURE 1.1 – Extrait d'un rapport sur la *bêta curve* du SX5E en hebdomadaire.

davantage, cohérent avec l'intuition de *risk-premium* commun plus dominant. (iii) $\beta_T < 1$ sur les indices SPX/HSCEI/NKY aux maturités courtes/moyennes : sensibilité relative des DivSwaps aux chocs spot plus faible (structure de marché, fiscalité, pratiques de *buybacks*, composition sectorielle, microstructure des *quotes*).

1.2.3 Contrôle par données de marché

En mensuel sur SX5E, les estimations à partir de données de marché confirment la montée de la corrélation avec T ($\approx 0.43 \rightarrow 0.86$) et donnent des β_T proches de ceux trouvés précédemment ($\beta_{2Y} \approx 1.15$, $\beta_{5Y} \approx 1.07$, $\beta_{7Y} \approx 1.01$).

Conséquence opérationnelle. Cette *beta-curve* robuste, corroborée par des données de marché, pourrait être utilisée pour challenger les marquages des dividendes par le FO, en apportant une lecture par maturité fondée sur des régressions indépendantes.

Term	1Y	2Y	3Y	4Y	5Y
Correlation (R^2)	0.0001	0.0901	0.2978	0.3555	0.4023
Coeff	0.1886	1.7175	1.6278	1.3729	1.3192

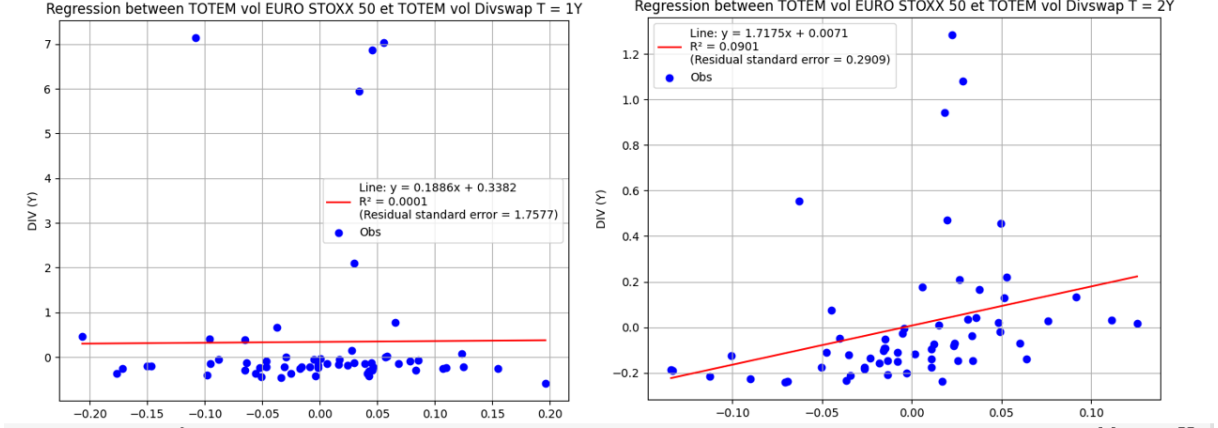


FIGURE 1.2 – Extrait d'un rapport sur la *bêta curve of vol* sur SX5E en mensuel.

1.3 Volatilité de dividendes : régressions et calibration SABR

1.3.1 Spécifications

On étudie la relation entre variations de volatilité de dividendes et variations de volatilité *Spot* :

$$\Delta\sigma_{T,t}^{\text{Div}} = a_T + b_T \Delta\sigma_t^{\text{Spot}} + u_{T,t}, \quad t \in \mathcal{T}, \quad (1.2)$$

avec deux définitions de $\sigma_{T,t}^{\text{Div}}$: (i) Volatilité implicite ATM d'options sur DivSwaps, (ii) Volatilité ATM issue d'un calibrage SABR du smile d'options de DivSwaps.

1.3.2 Résultats (Euro Stoxx 50, mensuel)

Volatilité implicite ATM. Le pouvoir explicatif R^2 est faible au court terme et progresse avec la maturité (p. ex. $R^2 \approx 0.01$ à 1Y, 0.16 à 3Y, 0.23 à 4Y, 0.27 à 5Y) ; le bêta de vol b_T se stabilise autour de 1.2–1.6 à 3Y–5Y.

R^2 reste bas aux maturités courtes et monte jusqu'à ≈ 0.30 à 5Y ($b_T \sim 1.16$ –1.50 selon T).

Volatilité implicite ATM (Marché). Même diagnostic qualitatif, avec R^2 très faible au court terme puis croissant ($R^2 \approx 0.0001$ à 1Y, 0.30 à 3Y, 0.36 à 4Y, 0.40 à 5Y) ; b_T dans une fourchette proche (~ 1.3 –1.7 à 3Y–5Y).

R^2 est quasi nul jusqu'à 3Y, puis progresse (≈ 0.22 à 4Y, ≈ 0.40 à 5Y) ; b_T converge vers 1.3–1.35 au long terme.

Volatilité ATM issue d’une calibration SABR. La calibration SABR du smile d’options de DivSwaps a été répliquée pour extraire une ATM SABR et relancer la régression $\Delta\sigma^{\text{Div}}$ vs $\Delta\sigma^{\text{Spot}}$.

1.3.3 Lecture, limites et décisions

- **Lecture.** Au court terme, $\Delta\sigma_{\text{spot}}$ explique peu $\Delta\sigma_{\text{div}}$. Le lien devient plus lisible à partir de 3Y–5Y, avec b_T autour de 1.3–1.6. La reprise de la méthode ATM via SABR ne renverse pas le diagnostic : elle stabilise l’estimation mais ne la rend pas déterminante aux maturités courtes.
- **Limites.** Taille d’échantillon (peu de points en options de dividendes), bruit de mesure (asynchronisme des spots), hétérogénéité cross-indices (SX5E vs SPX/HS-CEI/NKY), et sensibilité aux périodes de stress.
- **Décision.** La relation $\Delta\sigma_{\text{div}}-\Delta\sigma_{\text{spot}}$ est non robuste sur le court terme ; le signal est plus exploitable sur le long terme, mais nécessite plus de données et un modèle structure complémentaire.

1.3.4 Calibration SABR (description mathématique)

On calibre, pour chaque maturité T , un modèle SABR lognormal (convention Black) sur le *smile* de volatilité implicite des options de DivSwaps, à partir d’un ensemble de strikes $\{K_i\}_{i=1}^N$ et des vols de marché correspondantes $\{\sigma_i^{\text{mkt}}\}_{i=1}^N$. Le forward F est extrait du fichier de marché, et les strikes absolus sont obtenus depuis des strikes en pourcentage via $K_i = \text{StrikePercent}_i \times F/100$.

Paramètres et conventions. Les paramètres du modèle sont $(\alpha, \rho, \nu, \beta)$ avec $\beta \in [0, 1]$ (lognormal standard pour $\beta = 1$). Un *displacement* optionnel $b \geq 0$ permet de traiter des valeurs faibles ou négatives de F ou K en posant $F_b = F + b$, $K_b = K + b$.

Formules asymptotiques de Hagan (2002) en convention Black.

À la monnaie, ($K_b \simeq F_b$), l’approximation d’ordre 1 en T est

$$\sigma_B^{\text{ATM}}(F, T; \alpha, \rho, \nu, \beta, b) \approx \alpha F_b^{\beta-1} \left(1 + \frac{2-3\rho^2}{24} \nu^2 T + \rho \beta \nu \alpha T \frac{F_b^{\beta-1}}{4} \right),$$

ce qui se simplifie en $\sigma_B^{\text{ATM}} \approx \alpha (1 + \frac{2-3\rho^2}{24} \nu^2 T + \rho \nu \alpha \frac{T}{4})$ pour $\beta = 1$.

Hors monnaie, pour $\beta = 1$ (lognormal pur),

$$z = \frac{\nu}{\alpha} \ln\left(\frac{F_b}{K_b}\right), \quad x(z) = \ln\left(\frac{\sqrt{1-2\rho z + z^2} + z - \rho}{1-\rho}\right),$$

et la volatilité Black est approchée par

$$\sigma_B(F, K, T; \alpha, \rho, \nu, \beta=1, b) \approx \left(\alpha \frac{z}{x(z)} \right) \left(1 + \frac{2-3\rho^2}{24} \nu^2 T \right).$$

Pour $0 < \beta < 1$, on utilise la forme générale

$$z = \frac{\nu}{\alpha} (F_b - K_b) \frac{(F_b K_b)^{\frac{1-\beta}{2}}}{F_b^{1-\beta} - K_b^{1-\beta}}, \quad \sigma_B \approx \frac{\alpha}{(F_b K_b)^{\frac{1-\beta}{2}}} \frac{z}{x(z)} \left(1 + \frac{2-3\rho^2}{24} \nu^2 T \right),$$

avec $x(z)$ défini comme ci-dessus.

Problème de calibration (par maturité). Étant donnée la grille de strikes $\{K_i\}$ et les vols de marché $\{\sigma_i^{\text{mkt}}\}$, on cherche à minimiser l'erreur quadratique moyenne entre vols modèle et marché :

$$\min_{\alpha, \rho, \nu} \text{RMSE}(\alpha, \rho, \nu) \equiv \sqrt{\frac{1}{N} \sum_{i=1}^N \left(\sigma_B(F, K_i, T; \alpha, \rho, \nu, \beta, b) - \sigma_i^{\text{mkt}} \right)^2},$$

sous contraintes usuelles $\alpha > 0$, $\rho \in [-0.999, 0.999]$, $\nu > 0$. En pratique, on emploie une méthode de type L-BFGS-B⁸ avec *initial guess* $(\alpha_0, \rho_0, \nu_0)$ (par exemple α_0 autour du niveau ATM, $\rho_0 \simeq 0$, $\nu_0 \simeq 0.5$).

Volatilité ATM calibrée et surface. Pour chaque maturité T , on obtient les paramètres $(\alpha_T, \rho_T, \nu_T)$ calibrés, d'où la volatilité ATM calibrée $\sigma_B^{\text{ATM}}(F, T; \alpha_T, \rho_T, \nu_T, \beta, b)$. En répétant la calibration sur un ensemble de maturités $\{T_m\}$ (par exemple 12, 24, 36, 48, 60 mois), on construit une surface de volatilité

$$\left\{ \sigma_B(F, K_{i,m}, T_m; \alpha_{T_m}, \rho_{T_m}, \nu_{T_m}, \beta, b) \right\}_{m,i},$$

où $K_{i,m}$ sont les strikes de la grille au temps T_m . Cette surface calibrée est ensuite utilisée pour comparer la volatilité ATM issue de SABR à la volatilité ATM implicite du marché et pour lancer la régression $\Delta\sigma^{\text{Div}}$ vs $\Delta\sigma^{\text{Spot}}$.

8. La minimisation numérique est réalisée à l'aide de l'algorithme *L-BFGS-B* (*Limited-memory Broyden-Fletcher-Goldfarb-Shanno with Bounds*), une méthode quasi-Newton conçue pour l'optimisation non linéaire sous contraintes de bornes. Cet algorithme approxime la matrice hessienne inverse à partir d'un nombre limité d'informations issues des gradients successifs, ce qui le rend particulièrement efficace pour des problèmes de dimension modérée. La variante *B* permet d'imposer explicitement des contraintes de type *box* sur les paramètres, telles que $\alpha > 0$, $\rho \in [-1, 1]$ et $\nu > 0$, garantissant ainsi la faisabilité économique et mathématique des solutions obtenues.

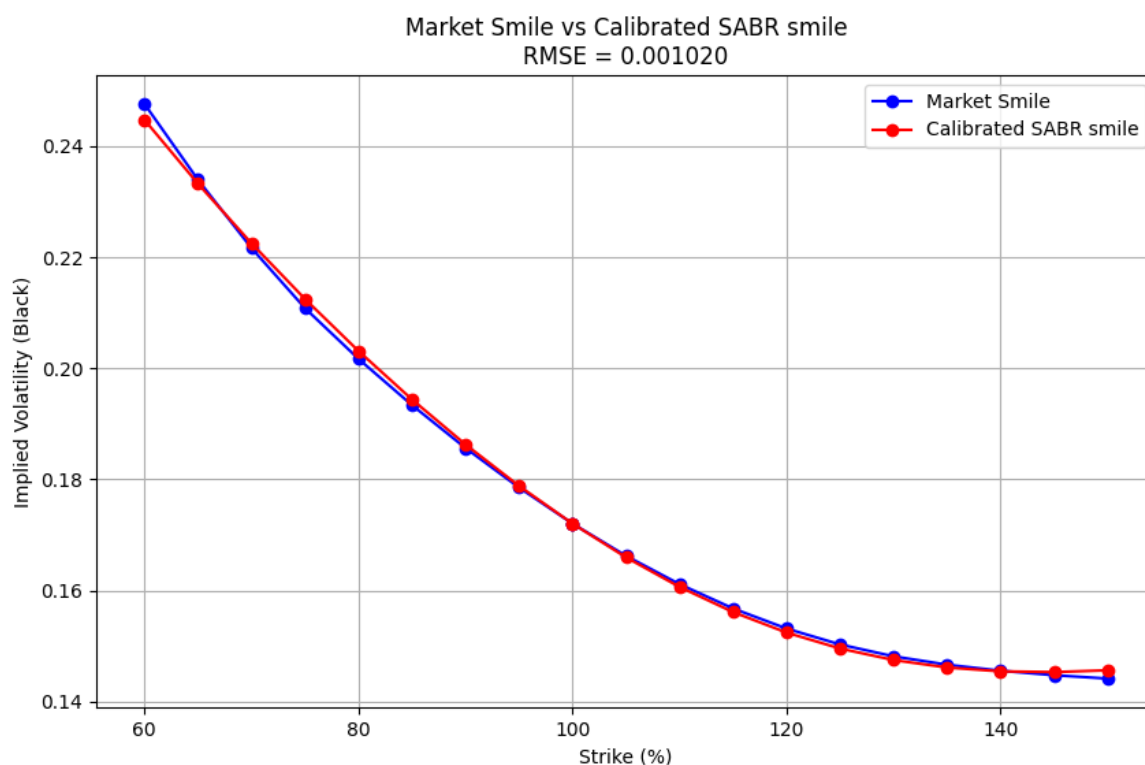


FIGURE 1.3 – Exemple de Calibration SABR sur le *Spot*.

Ouverture et Transition

Les limites empiriques de la calibration SABR classique, en particulier la difficulté à concilier simultanément une bonne représentation des ailes de volatilité et une résolution satisfaisante autour de la monnaie, ont conduit à l'étude de schémas de calibration pondérés (*SABR weighted*). Ces approches, qui privilégient les strikes les plus liquides, améliorent sensiblement la calibration autour de l'ATM, au prix d'une dégradation possible dans les ailes, mettant en évidence les contraintes de flexibilité intrinsèques au modèle. Dans ce contexte, l'analyse de la courbe de β par maturité et les constats relatifs à la structure de la volatilité des dividendes (avec reprise de la calibration SABR) constituent un point de départ naturel pour la suite, en vue de l'élaboration d'un cadre plus structurel pour la *term-structure* des dividendes et l'évaluation des choix de marquage.

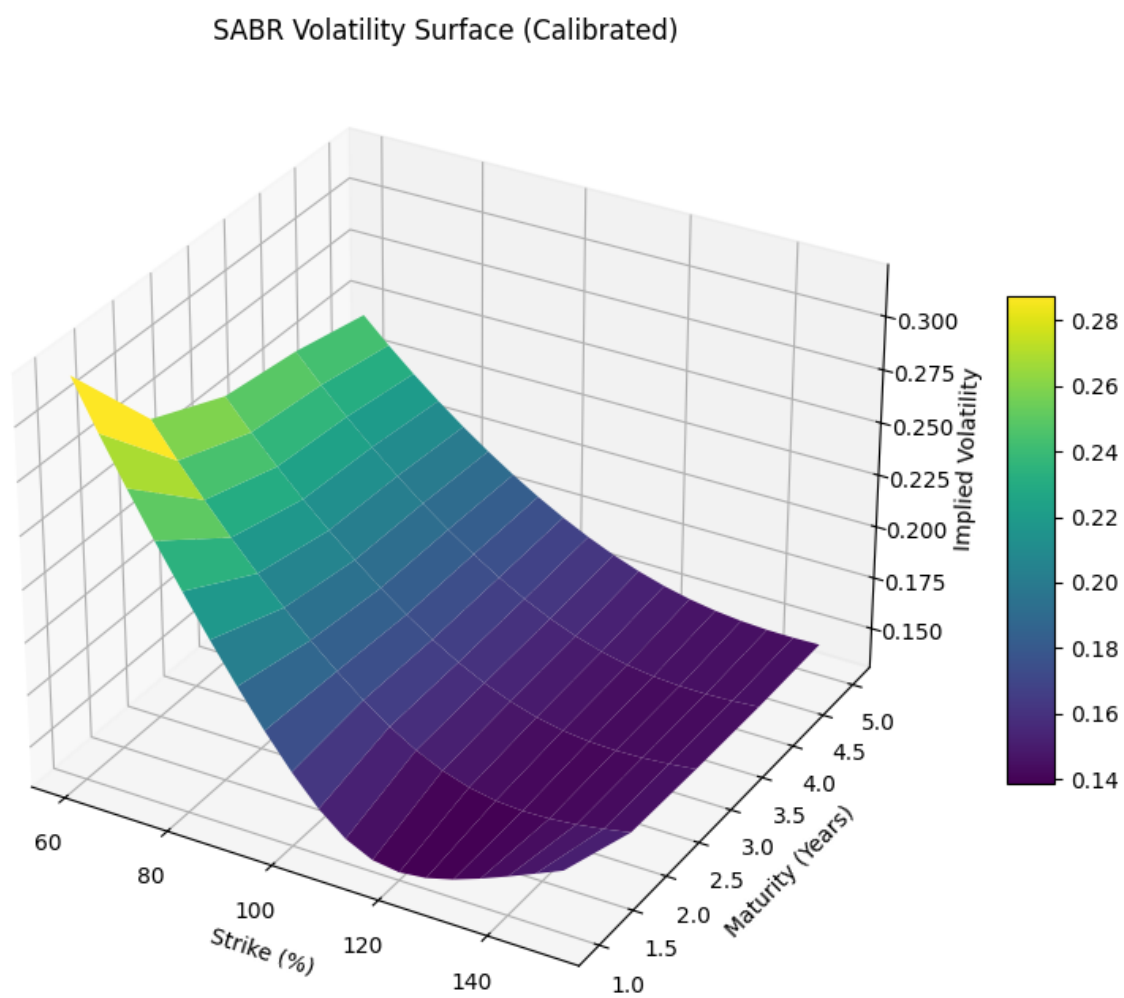


FIGURE 1.4 – Exemple de surface de vol SABR sur le *Spot*.

Chapitre 2

Stratégies sur dividendes : *mean-reversion* du bêta et *steepener* de pente

2.1 Motivation : pourquoi trader les dividendes ?

Au cours du stage, je me suis fortement intéressé aux Dividend Swaps et Dividend Futures et aux raisons de les trader :

- **Exposition de type equity** avec un profil de risque distinct : corrélation des dividendes au spot qui augmente avec la maturité ; faible volatilité aux très courts termes (dividendes annoncés), sensibilité plus forte au spot aux maturités longues. C'est exactement ce que nous avons observé sur les regressions linéaires à partir des données du marché. Le Beta en fonction de la maturité des divswaps.
- **Trades d'allocation** : mise en place de positions de type *Dividend Yield* (appariement de notionnels long div versus short equity) et *steepeners* (jouer la pente de la courbe des dividendes entre deux maturités).
- **Hedging et relative value** : couverture de produits structurés (par exemple autocalls), dispersion dividendes d'indice versus dividendes de titres individuels, combinaisons contre les taux d'intérêt ou l'inflation, TRS et cash-and-carry, avec rôle du repo et des bases.
- **Contraintes de marché** : liquidité hétérogène par maturité, spreads plus larges sur les longues échéances, effets de fiscalité et de buybacks sur les niveaux de forward et de dividendes.

Cette revue m'a conduit à tester deux stratégies quantitatives sur données historiques : une *mean-reversion* du bêta DivSwap/Spot et un *steepener* de la pente de bêta entre maturités.

2.2 Stratégie n°1 : *mean-reversion* du bêta DivSwap vs Spot

2.2.1 Idée et pipeline

L'idée centrale est de détecter des excès temporairement persistants du bêta de marché entre les rendements % d'un contrat de dividendes d'indice (*DivSwap*) et les rendements % du *spot*, puis de les traiter par un aller-retour bi-sens (*long/short DivSwap*) avec neutralisation instantanée au 1^{er} ordre par une jambe spot. Le bêta est estimé par maturité (*bucket*) et suivi dans le temps pour construire un signal de *mean-reversion* suffisamment robuste avant exécution.

Données et panel marché. Les séries nécessaires sont (i) les quotes DivSwap identifiées par *ExpiryDate* et *ValuationDate*, (ii) la série spot (niveau et date). On calcule les rendements % DivSwap par contrat et les rendements % spot, puis on fusionne les deux sur la date pour former un *panel* marché propre (filtrage, normalisation).

Buckets de maturité et bêta “de référence”. On discrétise l'axe des maturités T en intervalles réguliers (pas `step_years`) ; chaque observation tombe dans un *bucket* i centré sur T_i . Dans ce *bucket*, on estime le modèle linéaire (unités en %) :

$$\text{DivswapPct}_{d,e} = \alpha_i + \beta_i \text{SpotPct}_d + \varepsilon_{d,e}, \quad (d, e) \in B_i,$$

par OLS avec erreurs-types robustes (HC1). On retient le bêta $\hat{\beta}_i$ et son erreur-type $\text{SE}(\hat{\beta}_i)$, qui serviront de niveau et de dispersion de référence pour la suite.

Bêta roulant et construction du signal. Pour chaque bucket, on calcule un bêta roulant $\hat{\beta}_i(t)$ sur une fenêtre calendaire (`fine_window_weeks`), et on le normalise en z -score par rapport au bêta de référence du bucket :

$$z_i(t) = \frac{\hat{\beta}_i(t) - \hat{\beta}_i}{\sigma_i}, \quad \sigma_i = \text{SE}(\hat{\beta}_i)$$

Un filtre ADF sur les résidus du modèle de *bucket* valide la stationnarité avant *trading* ; en l'absence de stationnarité, le *bucket* est désactivé.

Règles d'entrée/sortie et choix du contrat. À la date t (fin de fenêtre), on entre si le signal franchit les seuils : $z_i(t) \leq -z_{\text{enter}}$ (*long DivSwap*) ou $z_i(t) \geq +z_{\text{enter}}$ (*short DivSwap*), sous garde-fou $\hat{\beta}_i(t) > 0$. On sélectionne l'*expiry* cotant le jour t dont la maturité est la plus proche de T_i . La sortie est déclenchée dès que $|z_i(t)| \leq z_{\text{exit}}$ (*reversion*) ou si la durée dépasse `max_weeks` (*timeout*).

Neutralisation instantanée et tailles. La jambe spot est dimensionnée au bêta d'entrée pour neutraliser le delta au 1^{er} ordre à l'instant t : si N_{div} est le nombre de contrats DivSwap et MV_{div} la valeur de marché associée, la valeur de marché de la jambe spot à l'entrée est $MV_{\text{spot}} \approx \hat{\beta}_i(t) \times MV_{\text{div}}$ (configuration `hedge_by_beta=True`). Des variantes *dollar-neutral* ou *vol-neutral* peuvent être activées, mais la configuration par défaut reste le *hedge par bêta d'entrée*.

Calcul du P&L jambe par jambe. À la sortie, on valorise chaque jambe à partir des prix de sortie et des tailles effectivement tenues (DivSwap et spot), puis on agrège :

$$\text{PnL}_{\text{amount}} = \text{PnL}_{\text{DivSwap}} + \text{PnL}_{\text{Spot}}, \quad \text{PnL}_{\text{return}} = \frac{\text{PnL}_{\text{amount}}}{\max(10^{-12}, MV_{\text{div, entry}})}.$$

Les conventions de signe dépendent de la direction (*long_div* vs *short_div*) ; un tableau *trades* consigne chaque opération (dates, tailles, raisons de sortie, *PnL* par jambe et total), et une synthèse par maturité agrège nombre de trades, taux de réussite, *PnL* cumulé et moyenne.

Robustesse. Le pipeline comporte des garde-fous : stationnarité des résidus (ADF), variance minimale de la fenêtre spot, seuils z calibrés par *bucket* (dispersion = erreur-type du bêta), *timeout*, et bornes sur tailles (point values, FX, cap sur unités spot). Les exports (`panel`, `beta_by_bucket`, `trades`, `summary`) facilitent l'audit et la réplication.

2.2.2 Optimisation du *pipeline* (score pénalisé, grille et recherche aléatoire)

Ce code met en place une optimisation opérationnelle de la stratégie DivSwap–Spot, en s'appuyant sur un *Evaluator* qui charge une seule fois le panel marché (lecture & fusion) et met en cache les estimations de bêta par *bucket* pour un couple (`step_years`, `min_obs_bucket`). L'évaluateur réutilise ce cache et relance la stratégie sur un intervalle de maturités $[T_{\min}, T_{\max}]$ en mesurant, pour chaque jeu de paramètres, des mesures agrégées : *PNL total*, *nb_trades*, *hit_ratio*, *reversion_rate*, *timeout_rate*, *avg_duration_days*, ainsi que le nombre de *buckets* traités.

Un **score pénalisé** est ensuite calculé pour privilégier les combinaisons robustes :

$$\text{score} = \text{PNL} - \left(w_{\text{rev}} [0.60 - \text{reversion_rate}]_+ + w_{\text{to}} [\text{timeout_rate} - 0.25]_+ + w_{\text{tr}} [20 - \text{nb_trades}]_+ \right),$$

où $[x]_+ = \max(0, x)$ et les poids $(w_{\text{rev}}, w_{\text{to}}, w_{\text{tr}})$ sont choisis grands pour faire respecter les contraintes cibles.

Deux **moteurs d'optimisation** sont fournis :

- *sweep_grid* (balayage grille) : parcourt systématiquement les combinaisons de paramètres à optimiser (*fine_window_days*, *min_window_obs*, *use_moving_zscore*, *moving_z_lookback_days*, *z_enter*, *z_exit*, *max_days*). Il affiche la progression (pourcentage, temps écoulé, ETA, *best-so-far*) et écrit en *CSV*.
- *random_search* : échantillonne aléatoirement les mêmes paramètres dans des plages spécifiées, avec la même logique d'affichage, de score et d'écriture.

Au final, l'optimisation classe les réglages qui maximisent le *PNL* et respectent les contraintes de robustesse (taux de réversion, *timeouts*, volume de trades), tout en garantissant la reproductibilité via cache, journaux de progression et exports CSV. Voici la grille utilisé et la synthèse des résultats.

```
# C) Grille "coarse" (raisonnable en temps)
grid = {
    "step_years": [4.0/12.0, 6.0/12.0],
    "fine_window_weeks": [12, 20, 30],
    "min_window_obs": [15, 25],
    "z_enter": [1.2, 1.5, 1.8, 2.0],
    "z_exit": [0.25, 0.35, 0.45],
    "max_weeks": [40, 52, 70]
}
```

FIGURE 2.1 – Grille de paramètres pour optimiser les paramètres de la stratégie

	pnl_total	nb_trades	hit_ratio	reversion_rate	timeout_rate	guard_negative_rate	avg_duration_days	n_buckets	runtime_sec
SUM(GRID)	56534.578	5702	243.63935	402.6663266	29.33367337	0	11847.90536	7128	3433.03334
MOY(GRID)	130.867079	13.19907407	0.56398	0.932097978	0.067902022	0	27.42570685	16.5	7.946836434
		PnL/Trade	MV/Trade	Rendement	Rendement %				
		9.914868125	1800	0.00550826	0.550826007				

	pnl_total	nb_trades	hit_ratio	reversion_rate	timeout_rate	guard_negative_rate	avg_duration_days	n_buckets	runtime_sec
SUM(Rand)	4471.985027	669	26.38918733	28.57375524	18.42624476	0	1025.829626	953	314.1932213
MOY(rand)	89.43970055	13.38	0.527783747	0.571475105	0.368524895	0	20.51659252	19.06	6.283864427
		PnL/Trade	MV	Rendement	Rendement %				
	6.684581506	1800	0.003713656	0.371365639					

FIGURE 2.2 – Résultats de GRID et du RandomSearch

Ces résultats sont très concluants, on arrive à un rendement moyen de 0.5% par trade. Cependant, il ne fallait pas crier victoire trop vite...

2.2.3 Validation par apprentissage automatique (train, validation, test)

Dans un premier temps, les backtests donnaient un PnL positif et stable pour de nombreux réglages (pas de maturité, fenêtres, seuils d'entrée/sortie, durée maximale). Pour vérifier que cette performance tenait hors échantillon, j'ai mis en place une validation par apprentissage automatique en trois phases (sans modifier le moteur de backtest) :

Périmètres temporels.

- **Entraînement** : année 2023 (apprentissage des hyperparamètres sur données historiques),
- **Validation** : année 2024 (sélection robuste des hyperparamètres),
- **Test** : janvier–novembre 2025 (évaluation finale hors échantillon, sans réajustement).

Grille et métriques. Une grille compacte (environ 64 combinaisons) explore les hyperparamètres clés : pas de maturité (*step years*), longueur de fenêtre fine (en semaines), minimum d'observations dans la fenêtre, seuils d'entrée/sortie (z_{enter} , z_{exit}), durée maximale du trade. Pour chaque combinaison, le *backtest* retourne des mesures agrégées : PnL total, nombre de trades, taux de réussite, taux de réversion, taux de *timeout*, durée moyenne.

Sélection robuste (sur validation). La sélection impose des contraintes cibles : taux de réversion $\geq 60\%$, taux de *timeout* $\leq 25\%$, nombre de trades ≥ 20 . Si aucune combinaison ne les satisfait, un *fallback* affine la grille (fenêtre plus courte, seuils plus souples), puis un mode *lite* (contraintes moins strictes). En dernier recours, une sélection *PnL-only* (meilleur PnL sur la période de validation) est utilisée.

Évaluation finale (hors échantillon). Les hyperparamètres retenus sont figés, puis appliqués tels quels sur la période de test (2025). Des diagnostics supplémentaires ventilent le PnL par jambe (dividendes vs spot), par direction, et par intervalle de maturité.

Conclusion de l'exercice. Malgré des PnL positifs en entraînement et, parfois, en validation, la performance hors échantillon ne se maintient pas : le PnL se dégrade et révèle une dépendance résiduelle au spot. Autrement dit, la stratégie apprise sur une période ne généralise pas convenablement : le signal exploité est instable et "spot-orienté", confirmant qu'il ne s'agit pas d'une prime robuste et delta-neutre.

2.2.4 Pourquoi la stratégie ne fonctionne pas.

Malgré la neutralisation instantanée au moment de l'entrée (*hedge spot* dimensionné sur le bêta du jour), le $P\&L$ sur l'horizon du trade reste piloté par un terme d'interaction entre la dérivée du bêta et le mouvement du spot. Autrement dit, la performance devient fonction du chemin (variance et timing du spot), d'où une directionnalité résiduelle.

Notations et mise en place. À l'entrée t_e , on prend une jambe dividende (DivSwap) de valeur de marché $P_{2,e}$ et un *hedge spot* de valeur $P_{1,e}$ dimensionné au bêta d'entrée β_e (neutralisation instantanée de premier ordre). Entre t_e et t_x :

- S : niveau de l'indice spot ; $\Delta S \equiv S_x - S_e$,
- P_2 : valeur de la jambe DivSwap ; P_1 : valeur de la jambe spot (*hedge*),
- β : pente DivSwap % contre Spot % (estimée en régression) ; $\Delta\beta \equiv \beta_x - \beta_e$,
- λ : facteur de mise à l'échelle du *hedge* (lié au choix de neutralisation par bêta d'entrée),
- ε : résidu (bruit d'agrégation, base, frictions, discrétisation).

Développement de premier ordre (intuition). Sur une période courte, on linearise le $P\&L$ agrégé de la position (DivSwap + *hedge spot*). En notant que le *hedge* est figé au bêta d'entrée et que le bêta dérive ensuite, on obtient, à un facteur d'échelle près¹ :

$$\Delta\Pi \approx \underbrace{\Delta\beta_2 P_2}_{\text{Variation de sensibilité DivSwap}} - \underbrace{\lambda \Delta\beta_1 P_1 \Delta S}_{\text{Terme d'interaction } (\Delta\beta \times \Delta S) \text{ lié au hedge figé}} + \underbrace{\varepsilon}_{\text{Base, frictions, discrètes}}. \quad (2.1)$$

Lecture financière de chaque terme.

- *Variation de sensibilité DivSwap* ($\Delta\beta_2 P_2$). Même si le spot ne bougeait pas, un changement de régime dans la relation DivSwap–Spot (bêta qui glisse) "re-tarife" la jambe dividende. Ce terme existe parce que l'on trade un instrument de niveau de prix avec une sensibilité statistique qui n'est pas constante dans le temps.
- *Terme d'interaction* ($-\lambda \Delta\beta_1 P_1 \Delta S$). Le *hedge spot* a été calibré sur β_e puis laissé statique jusqu'à la sortie. Dès que β dérive ($\Delta\beta \neq 0$), la neutralité initiale se décale : on récupère alors un résiduel directionnel proportionnel au produit $\Delta\beta \times \Delta S$. C'est la source principale de dépendance au chemin du spot.
- *Résidu* (ε). Différences de base DivSwap–Spot, spreads, liquidité, discretisation des fenêtres, etc.

1. L'idée est de séparer l'impact purement niveau (via P_j) de l'impact sensibilité (via $\Delta\beta$) et du *carry/roll*. Les constantes de proportionnalité dépendent des conventions de point value et du choix d'unités.

Lien avec la mise en œuvre. La mécanique de la stratégie correspond exactement à cette décomposition :

- Le signal est un *z-score* sur le bêta roulant par bucket (entrée bi-sens quand $|z|$ franchit le seuil ; sortie par reversion ou *timeout*).
- Le *hedge* est dimensionné au bêta d'entrée et non ré-équilibré intra-trade : toute dérive $\Delta\beta$ crée le terme d'interaction avec ΔS .
- Le *P&L* jambe par jambe agrégé met en évidence, empiriquement, la domination des composantes *carry/roll* interaction ($\Delta\beta \times \Delta S$) sur le gain « mean-reversion » attendu.

Conclusion. Même si l'entrée est instantanément neutre au spot, la dérive du bêta et le *roll* de la courbe rendent la position "spot-orientée" et non delta-neutre dans la durée. La stratégie n'est donc pas viable sans prime structurelle indépendante du *spot* et du *carry*.

2.3 Stratégie n°2 : *steepener* de la pente de bêta (maturité constante)

2.3.1 Idée et implémentation

À chaque fin de mois, on sélectionne deux contrats DivSwap en maturité constante (par exemple court ≈ 2 ans, long ≈ 5 ans), on estime les bêtas court/long sur fenêtre, on les lisse (moyenne exponentielle), puis on construit la pente :

$$\text{Slope}(t) = \beta_{\text{long}}^{\text{smooth}}(t) - \beta_{\text{short}}^{\text{smooth}}(t).$$

Pour éviter qu’une jambe domine, on égalise les “beta-dollars” via

$$\lambda^* = \frac{\beta_{\text{long}} \cdot P_{\text{long}}}{\beta_{\text{short}} \cdot P_{\text{short}}} \quad \text{et} \quad N_{\text{court}}^{\text{eff}} = N_{\text{court}}^{\text{base}} \cdot \lambda^*,$$

puis on met un *hedge spot* partiel ($\alpha \in (0, 1)$) pour une neutralisation de premier ordre. Un *gating de carry* (*proxy de drift sur lookback*) autorise ou non l’ouverture du trade mensuel. Un journal de backtest conserve ouvertures/fermetures, tailles et P&L.

2.3.2 Pourquoi la stratégie ne fonctionne pas.

Malgré l’ingénierie mise en place (maturité constante, égalisation des “beta-dollars”, *hedge* partiel, lissage des bêtas), le *PnL* obtenu reste dominé par le *carry* et le *roll* de la courbe des prix de dividendes en régime *contango*. En pratique, c’est un mismatch de mesure : la pente de bêta est une statistique *sous P* (pente de rendements), alors que les prix des dividend swaps/futures reflètent des espérances *sous Q* (risk-neutre) avec des primes d’horizon spécifiques. Un signal sur la pente des bêtas ne se transforme pas directement en prime monétisable sur la courbe de prix.

L’erreur conceptuelle a été de vouloir trader la pente du bêta entre rendements de dividendes et rendements de spot (“beta term structure”) alors que la stratégie de *steepener* s’adresse en réalité à la courbe de dividendes elle-même (“dividend term structure”), c’est-à-dire aux niveaux et *spreads* de prix par maturité (et leur *carry*), pas aux pentes de sensibilités en rendements. Autrement dit, il fallait cibler des écarts de niveaux sur la term-structure des dividendes (*growth/spread* de dividendes par maturité), plutôt que la pente des bêtas *sous P* ; c’est cette confusion qui explique l’absence de performance robuste hors échantillon.

Renvoi aux annexes

Les documents détaillant les *pipelines*, paramètres, équations et journaux de *backtests* sont fournis en annexes (stratégie mean-reversion DivSwap–Spot, stratégie steepener bêta en maturité constante, revue des usages et pratiques de marché sur dividendes).

Transition

Ces échecs contrôlés ont été formateurs : ils soulignent l’importance de la structure de prix (*carry* et *roll* de courbe) et d’une modélisation sous Q des dividendes. Le chapitre suivant présentera un modèle de structure de dividendes pour construire une term-structure cohérente et exploitable, utile à la fois pour le *challenging* des marquages et pour l’analyse de primes d’horizon.

Chapitre 3

Modèle de structure par terme des dividendes (DTS)

3.1 Vue d'ensemble et idée directrice

L'objectif est de reconstruire et d'interpréter la structure par terme des dividendes à partir des prix de dérivés de dividendes (Dividend Swaps/Dividend Futures) par maturité, en travaillant en valeurs présentes. Le cœur économique repose sur une variable unique qui agrège croissance, taux et inflation

$$g_t = \Delta D_t - r_t - \pi_t,$$

et sur l'idée qu'à court et moyen terme, deux facteurs latents *mean-reverting* (actualité vs. cycle) suffisent à expliquer la forme de la courbe et sa dynamique.

Glossaire des symboles :

- $P_t^{\text{exp}}(n)$: prix du dérivé dividendes à expiration $t+n$ (en années).
- $P_{n,t}^{\text{CM}}$: prix *constant-maturity* (lissage entre n et $n+1$).
- $\tau_{n,t}$: maturité exacte (en années) vers le 3^e vendredi de décembre de l'année $t+n$.
- $r(\tau)$: taux sans risque (interpolé) pour la maturité τ ; $PV_{n,t}$: valeur présente.
- $y_{n,t} = \ln PV_{n,t} - \ln PV_{1,t}$: mesure (log-ratio) par maturité.
- $x_t, \tilde{x}_t$: facteurs latents (court terme / moyen terme).
- $H(n)$: charges par maturité (pondérations qui décroissent avec n).

3.2 Étape 1 — Maturités constantes et Actualisation

Maturité exacte et prix *constant-maturity*. Pour l'Eurostoxx 50, l'expiration des dérivés dividendes est le troisième vendredi de décembre de l'année d'expiration $(t+n)$; la maturité exacte vaut

$$\tau_{n,t} = \frac{\text{expiry}(t+n) - t}{365.25}.$$

Afin d'éliminer les bosses liées au calendrier et à la saisonnalité de versement, on lisse entre deux années adjacentes :

$$P_{n,t}^{\text{CM}} = (1 - w_t) P_t^{\text{exp}}(n) + w_t P_t^{\text{exp}}(n+1), \quad w_t \in [0, 1].$$

Définition précise du poids saisonnier w_t . Le terme w_t représente le pourcentage cumulé de dividendes déjà versés dans l'année en cours. On le construit à partir de poids mensuels $\{w_m\}_{m=1}^{12}$ (profil hardcodé, conforme aux pratiques de place où la majorité des dividendes sont détachés entre avril et juin) :

$$w_t = \sum_{k=1}^{m-1} w_k + w_m \cdot \frac{\text{jour}(t)}{\text{jours_dans_le_mois}(t)},$$

où m est le mois de la date t . Intuition : plus les dividendes de l'année se sont écoulés, plus $P_{n,t}^{\text{CM}}$ pèse l'année suivante $(n+1)$; ce lissage n'existe pas sur le marché (qui cote des expirations fixes), mais il est essentiel pour l'analyse de la courbe en supprimant les bosses de calendrier.

Actualisation en valeur présente. On travaille sur la courbe de taux sans risque $r(\tau)$ et l'on passe en valeur présente :

$$PV_{n,t} = P_{n,t}^{\text{CM}} \exp\left(-r(\tau_{n,t}) \tau_{n,t}\right).$$

Courbe des taux 1Y–10Y et interpolation. Dans les données disponibles, nous ne disposons que de $r(\tau)$ aux nœuds $\{1Y, 2Y, 5Y, 7Y, 10Y\}$ (extraits directement du site officiel de la BCE¹). Pour obtenir une courbe complète aux années entières 3Y, 4Y, 6Y, 8Y, 9Y, puis dans un second temps lisser la courbe à toute maturité, nous avons procédé comme suit :

1. **Log-linéaire sur les *discount factors*** (années manquantes entières). On interpole $\ln DF(\tau)$ linéairement entre les nœuds disponibles, où $DF(\tau) = \exp(-r(\tau) \tau)$. *Pourquoi DF plutôt que r ?* Les *discount factors* sont multiplicatifs et concaves en τ ; une interpolation linéaire sur $\ln DF$ respecte mieux la convexité/monotonie des facteurs d'actualisation et limite les artefacts. Cette pratique est standard en

1. <https://data.ecb.europa.eu/data/datasets/>

taux pour éviter les biais sur les présents. On récupère ensuite les taux aux années entières par

$$r(\tau) = -\frac{\ln DF(\tau)}{\tau}, \quad \tau \in \{3, 4, 6, 8, 9\} \text{ ans.}$$

2. **Linéaire sur les taux** $r(\tau)$ (maturités décimales τ non entières). Une fois la grille annuelle $r(1Y), \dots, r(10Y)$ complétée, on calcule $r(\tau)$ pour toute $\tau \in [1, 10]$ par interpolation linéaire (équivalent de `np.interp`) entre les points annuels adjacents. Cela fournit directement $r(\tau)$ pour la maturité exacte $\tau_{n,t}$ (en années réelles), puis $DF(\tau) = \exp(-r(\tau)\tau)$ pour l'actualisation.

Remarques de robustesse. (i) *Clamp* aux bornes $[1Y, 10Y]$ pour éviter l'extrapolation hors grille; (ii) alignement strict des dates entre PV et courbe $r(\tau)$ (jointure *inner*); (iii) contrôle des monotonicités $DF(\tau) \downarrow$ et $r(\tau)$ raisonnable (pas de bosses artificielles).

Sanity checks (rappel). À date t fixée, $\{\tau_{n,t}\}$ doit croître; $P_{n,t}^{\text{CM}}$ est borné entre $P_t^{\text{exp}}(n)$ et $P_t^{\text{exp}}(n+1)$; $PV_{n,t}$ décroît avec τ et $r(\tau)$. Le lissage via w_t supprime les bosses intra-annuelles et rend la lecture de la structure par terme homogène avant passage aux mesures de l'étape 2.

3.3 Étape 2 — Mesures par maturité (log-ratios)

Définition. À partir des $PV_{n,t}$, on fabrique pour chaque date t des mesures de forme de courbe :

$$y_{n,t} \equiv \ln PV_{n,t} - \ln PV_{1,t}, \quad n \geq 2.$$

Ainsi, $y_{n,t}$ est un log-ratio sans dimension : stable numériquement, comparable dans le temps, et insensible au niveau global (base $n=1$).

Pourquoi cette forme ? Elle capte uniquement la structure par terme (forme relative par maturité), c'est cohérent avec le cadre de présent-valeur et la décomposition par horizons : la première maturité sert d'ancrage ; on ne modélise que les écarts de présents entre n et 1.

3.4 Étape 3 — 2 facteurs latents et filtre de Kalman

État latent et dynamique (rappel). On pose un état de dimension 2 (court terme x_t et moyen terme $\tilde{x}_t$) :

$$\begin{bmatrix} x_t \\ \tilde{x}_t \end{bmatrix} = \underbrace{\begin{bmatrix} \phi_1 & 1 - \phi_1 \\ 0 & \phi_2 \end{bmatrix}}_F \begin{bmatrix} x_{t-1} \\ \tilde{x}_{t-1} \end{bmatrix} + \underbrace{\begin{bmatrix} 0 \\ (1 - \phi_2) \bar{x} \end{bmatrix}}_c + \underbrace{\begin{bmatrix} w_{1,t} \\ w_{2,t} \end{bmatrix}}_{w_t}, \quad w_t \sim \mathcal{N}(0, Q), \quad Q = \text{diag}(\sigma_1^2, \sigma_2^2).$$

Les coefficients mémoire $\phi_1, \phi_2 \in (0, 1)$ contrôlent la vitesse de retour à la moyenne (lien demi-vie : $\phi \approx e^{-1/t_{1/2}}$).

Équations de mesure et charges par maturité (rappel). Pour le vecteur d'observations y_t (les $y_{n,t} = \ln PV_{n,t} - \ln PV_{1,t}$ empilés), on prend :

$$y_t = H_t \begin{bmatrix} x_t \\ \tilde{x}_t \end{bmatrix} + v_t, \quad v_t \sim \mathcal{N}(0, R), \quad R = \sigma_\varepsilon^2 I.$$

Paramétrisation lisse de type Nelson–Siegel : $b_1(n) = e^{-n/L_1}$, $b_2(n) = 1 - b_1(n)$, et $H(n) = [b_1(n), b_2(n)]$.

Construction de la matrice de transition F

F dicte l'évolution naturelle des facteurs du jour $t-1$ à t (mémoires ϕ_1, ϕ_2 et bascule court $\rightarrow$ moyen). Le CT gravite autour du LT.

$$F = \begin{bmatrix} \phi_1 & 1 - \phi_1 \\ 0 & \phi_2 \end{bmatrix}.$$

Code : `np.array([[phi1,(1-phi1)], [0, phi2]])` construit F à partir des demi-vies.

Vecteur de *drift* c

c est une force de rappel vers un niveau de long terme $\bar{x}$ (stabilité macro, cible d'inflation/croissance), évitant un effondrement vers 0.

$$c = \begin{bmatrix} 0 \\ (1 - \phi_2) \bar{x} \end{bmatrix}.$$

Code : `np.array([0, (1-phi2)*xbar])` retourne c .

Covariance de processus Q

Q injecte à chaque pas de temps le risque intrinsèque des facteurs (chocs court et cycle).

$$Q = \text{diag}(\sigma_1^2, \sigma_2^2).$$

Code : `np.diag([sigma1**2, sigma2**2])` crée Q .

Prédiction de l'état (a priori)

`x_pred` est la meilleure estimation avant ouverture du marché à t , juste avec la dynamique interne.

$$\hat{x}_{t|t-1} = F \hat{x}_{t-1|t-1} + c.$$

Code : `x_pred = F @ x_pred + c` calcule $\hat{x}_{t|t-1}$.

Prédiction de l'incertitude (a priori)

`P_pred` quantifie le doute associé à la prédiction a priori (propagation d'erreur).

$$P_{t|t-1} = F P_{t-1|t-1} F^\top + Q.$$

Code : `P_pred = F @ P_pred @ F.T + Q` calcule $P_{t|t-1}$.

Innovation

`innov` confronte le modèle au marché réel (surprise entre observation et prédiction projetée). S'il est supérieur à zero, le marché est plus cher que prévu.

$$\nu_t = y_t - H_t \hat{x}_{t|t-1}, \quad S_t = H_t P_{t|t-1} H_t^\top + R.$$

Code : `innov = y_t - H_t @ x_pred` retourne ν_t .

Gain de Kalman (confiance)

Le facteur de Kalman indique de combien on doit corriger l'état avec la surprise (pondération optimale). Si R est significativement grand, K est égal à zéro donc le modèle ne change pas d'avis. Si la prédiction de l'incertitude (P -pred) est significativement grande, $K = 1$ et le modèle change d'avis et utilise le marché pour se recalculer.

$$K_t = P_{t|t-1} H_t^\top S_t^{-1}.$$

Code : `K = P_pred @ H_t.T @ np.linalg.pinv(S)` calcule K_t .

Mise à jour de l'état (a posteriori)

x_f corrige la prédiction initiale d'une fraction de la surprise.

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t \nu_t.$$

Code : $x_f = x_{pred} + K @ innov$ produit $\hat{x}_{t|t}$.

Mise à jour de l'incertitude (récompense d'info)

L'objectif ensuite est de réduire la variance *a priori* une fois l'information intégrée (on est moins incertain). Car plus on a de confiance K_t , moins on a d'incertitude $P_{t|t}$.

$$P_{t|t} = (I - K_t H_t) P_{t|t-1}.$$

Code : $P_f = (np.eye(dim_x) - K@H_t) @ P_{pred}$ fournit $P_{t|t}$.

Gain de lissage (rétrospection)

On revisite le passé éclairé par les prochaines observations, détecter les fausses alertes, stabiliser le cycle.

$$J_t = P_{t|t} F^\top P_{t+1|t}^{-1}.$$

Code : $J = P_f @ F.T @ np.linalg.inv(P_p)$ calcule J_t pour l'État lissé.

État lissé

On fait une combinaison optimale avant/après pour l'état final lissé. C'est une correction retrospective pour lisser les faux signaux (courbe historique propre).

$$\hat{x}_{t|T} = \hat{x}_{t|t} + J_t (\hat{x}_{t+1|T} - \hat{x}_{t+1|t}), \quad P_{t|T} = P_{t|t} + J_t (P_{t+1|T} - P_{t+1|t}) J_t^\top.$$

Code : $xs_smooth[t] = xs_filt[t] + J @ (xs_smooth[t+1] - x_preds[t+1])$ renvoie $\hat{x}_{t|T}$.

Log-vraisemblance gaussienne

L'objectif est d'évaluer la qualité d'ajustement et de piloter l'optimisation des hyperparamètres. Le calcul effectué jour par jour puis additionné. Chaque jour t , les erreurs innovations ν_t sont censées être gaussiennes.

$$\ell_t = -\frac{1}{2} (\nu_t^\top S_t^{-1} \nu_t + \ln |S_t| + m \ln 2\pi), \quad LL/obs = \frac{1}{T} \sum_t \ell_t.$$

Code : `ll_inc = -0.5* (innov.T @ np.linalg.solve(S,innov)`
`+ np.linalg.slogdet(S)`
`+ len(nlist)*np.log(2*np.pi))` accumule la LL normalisée.

Le meilleur modèle est celui qui combine précision et confiance.

Grid search

Pourquoi : on souhaite trouver une configuration robuste sans recourir à des gradients (pédagogie, stabilité, risque de valeurs aberrantes ou plantage.). On teste une grille de valeurs de paramètres et trouve la meilleure combinaison.

Protocole : grilles sur demi-vies (court, moyen), variances $(\sigma_1, \sigma_2, \sigma_\varepsilon)$, niveau long terme $\bar{x}$; pour chaque point, filtrage + lissage + LL/obs, conserver le meilleur.

Launch le balayage et résume LL/obs, MAE par maturité.

Benchmark 1 facteur

Le but est d'objectiver l'apport du second facteur en le comparant à la performance du modèle à 1 facteur.

Modèle : $x_t = \phi x_{t-1} + w_t$, (x_t n'est pas un vecteur de dimension 2 mais un simple chiffre, F est alors un simple retour à la moyenne)

$y_t = h(n) x_t + v_t$, $h(n) = e^{-n/L_1}$. Le modèle est incapable de voir une courbe qui s'inverse.

Diagnostics : comparer LL/obs et MAE par maturité entre 1f et 2f ; inspecter où le 2f baisse l'erreur.

Code : A la fin, on compare `best_ll` avec `best_ll_1f`.

3.4.1 Diagnostics et sorties

Pourquoi. Évaluer la qualité d'ajustement du modèle (2 facteurs vs 1 facteur), quantifier l'erreur par maturité, mesurer la vraisemblance moyenne, et exporter des séries/fichiers pour audit et reproduction.

Reconstructions et erreurs. Pour chaque maturité n et chaque jour t , on reconstruit

$$\hat{y}_{n,t} = H(n) \begin{bmatrix} x_t \\ \tilde{x}_t \end{bmatrix},$$

puis l'erreur absolue $e_{n,t} = |y_{n,t} - \hat{y}_{n,t}|$.

MAE par maturité et moyenne. Le *MAE* d'une maturité n (*maturity average error*) est défini comme la somme des erreurs absolues sur l'horizon observé,

$$\text{MAE}_n = \sum_t |y_{n,t} - \hat{y}_{n,t}|,$$

et l'indicateur global

$$\text{MAE}_{\text{mean}} = \frac{1}{N} \sum_{n=1}^N \text{MAE}_n,$$

où N est le nombre de maturités. (Au besoin, on peut aussi rapporter une version moyenne par jour en divisant chaque MAE_n par le nombre de dates observées.)

Log-vraisemblance par observation. La log-vraisemblance totale est accumulée quotidiennement dans la boucle de filtrage :

$$\text{loglik_total} = \sum_t \text{ll_inc}(t), \quad \text{ll_per_obs} = \frac{\text{loglik_total}}{\text{n_obs_total}}.$$

À chaque date t , l'incrément gaussien $\text{ll_inc}(t)$ est celui du filtre de Kalman (innovation ν_t , covariance S_t).

Exports CSV.

- États lissés : fichier avec les colonnes `x_short` et `x_medium` (une ligne par jour).
- Résumé paramètres et erreurs : fichier contenant les paramètres gagnants (demi-vies, $\sigma_1, \sigma_2, \sigma_\varepsilon, \bar{x}$), les MAE par maturité et `MAE_mean`.

Résultats empiriques (principaux chiffres). On observe `MAE_mean` = 0.0192933, soit $\approx 1.93\%$ (en log, interprété comme une erreur moyenne d'environ 1.93% lorsque le modèle reconstruit la courbe des dividendes). Le modèle colle bien à la réalité sans sur-apprendre.

La log-vraisemblance moyenne est `ll_per_obs` = 0.6216 pour le modèle 2 facteurs, contre -89.04 pour le modèle 1 facteur. De plus, la variance des erreurs (par maturité) est très faible pour le 2 facteurs, et nettement plus élevée pour le 1 facteur. C'est une victoire écrasante du 2 facteurs sur le 1 facteur : le gain de log-vraisemblance indique un ajustement bien supérieur, et la faible variance des résidus confirme la stabilité de l'explication par maturité.

Pourquoi ce chiffre n'a de sens qu'en comparaison. La log-vraisemblance est relative à un modèle et à une échelle d'observation (hypothèses gaussiennes, dimension du vecteur d'observation, normalisation). Sa valeur absolue est peu interprétable isolément ; c'est la comparaison entre modèles (ici 2 facteurs vs 1 facteur) qui établit la supériorité statistique et économique de l'un sur l'autre. En complément, la diminution de MAE_n et de sa variance par maturité corrobore la qualité de l'ajustement.

3.5 Étape 4 — Diagnostics et visualisations

DTS moyenne (parts de valeur présente). Pour chaque date t , on normalise $PV_{n,t}$ en parts qui somment à 1 :

$$\text{share}_{n,t} \equiv \frac{PV_{n,t}}{\sum_{k=1}^N PV_{k,t}}, \quad n = 1, \dots, N,$$

puis on moyenne temporellement $\text{share}_{n,t}$ pour obtenir la DTS moyenne (lecture : valeur plus *front-loaded* vs. plus étalée sur moyen/long terme).

Reconstruction et résidus. Avec $H(n)$ et les états lissés,

$$\hat{y}_{n,t} = H(n) \begin{bmatrix} x_t \\ \tilde{x}_t \end{bmatrix}, \quad e_{n,t} = y_{n,t} - \hat{y}_{n,t}.$$

On suit $MAE_n = \mathbb{E}_t[|e_{n,t}|]$ et $Vol_n = \sqrt{\text{Var}_t(y_{n,t})}$: petites MAE partout $\Rightarrow$ deux facteurs expliquent bien la forme ; Vol_n décroissante avec $n \Rightarrow$ maturités longues plus stables.

Proxies lisibles. Croissance 1 an $\simeq x_t$ (actualité) ; Forward 4 ans $\simeq \hat{y}_{5,t}/4$ (cycle annualisé) : deux séries parlantes.

3.6 Étape 5 — Ratio prix/dividende implicite et nowcasting

Objectif et principe. Vérifier si le modèle explique pourquoi l'indice boursier monte ou baisse chaque jour. Nous décomposons la variation quotidienne du prix en trois briques : (i) le *cash* d'aujourd'hui (dividende de l'année courante), (ii) les anticipations futures condensées par le modèle (ratio implicite P/D), (iii) le taux court (BCE, proxy 1 an). En agrégeant ces trois composantes, nous reconstruisons la courbure des variations du prix de l'indice avec un pouvoir explicatif d'environ **60%** ($R^2 \approx 0.603$).

Ratio implicite P/D (modélisé). Le ratio P/D synthétise « tout le reste de la courbe » au-delà de 1 an : c'est la somme des dividendes futurs anticipés par le modèle, rapportée au dividende d'aujourd'hui. À partir des reconstructions $\hat{y}_{n,t}$ et d'un niveau long terme $\bar{x}$,

$$dPD_t \equiv \sum_{n=2}^N \exp(\hat{y}_{n,t} + n\bar{x}), \quad \log PD_t = \ln dPD_t.$$

Lecture : si $\log PD_t$ monte, le modèle devient plus optimiste sur la croissance des dividendes futurs.

Code (construction scalaire) :

$$\text{pd_val} = \text{float}\left(\sum_{n \geq 2} \exp(\hat{y}_{n,t} + n\bar{x})\right).$$

Préparation des variations quotidiennes (rendements). On travaille en variations (rendements), et non sur les niveaux. Avec la fonction `diff` de `pandas` (ou différences finies) :

$$\Delta \ln P_t = \ln P_t - \ln P_{t-1} \quad (\text{indice}),$$

$$\Delta \ln D_t^{(1)} = \ln PV_{1,t} - \ln PV_{1,t-1} \quad (\text{dividende 1 an}),$$

$$\Delta r_{1Y,t} = r_{1Y,t} - r_{1Y,t-1} \quad (\text{taux 1 an, en points décimaux}),$$

$$\Delta \ln(PD_t) = \ln dPD_t - \ln dPD_{t-1} \quad (\text{changement d'opinion du modèle}).$$

Régression de *nowcasting* (journalier). Nous estimons une OLS sur données quotidiennes :

$$\Delta \ln P_t = \alpha + \beta_1 \Delta \ln D_t^{(1)} + \beta_2 \Delta r_{1Y,t} + \beta_3 \Delta \ln(PD_t) + \varepsilon_t.$$

Implémentation : on empile les variables explicatives $X = [\mathbf{1}, \Delta \ln D^{(1)}, \Delta r_{1Y}, \Delta \ln(PD)]$ et l'ordonnée $y = \Delta \ln P$ puis on résout $\min_{\beta} \|X\beta - y\|_2^2$ avec `numpy`.

$$\hat{\beta} = (X^\top X)^{-1} X^\top y, \quad R^2 = 1 - \frac{\sum_t (y_t - \hat{y}_t)^2}{\sum_t (y_t - \bar{y})^2}.$$

Résultats empiriques et lecture économique.

alpha	beta1	beta2	beta3	R2	N
0.000431	0.602643	2.858731	1.221977	0.604348	710

FIGURE 3.1 – Résultats du modèle sur une période de 10 ans

- **Puissance explicative.** $R^2 \approx 0.603$: $\sim 60\%$ de la volatilité quotidienne de l'indice est expliquée par $\Delta \ln D^{(1)}$, Δr_{1Y} et $\Delta \ln(PD)$. Avec seulement trois variables, le modèle explique plus de la moitié du bruit quotidien.
- **Ratio P/D (futur) dominant.** $\beta_3 \approx 1.22 > 1$: la variation de la structure des dividendes (brique PD) a l'impact le plus fort sur le prix ; le modèle est roi pour les anticipations.
- **Cash de l'année.** $\beta_1 \approx 0.60$: le dividende immédiat compte, mais le marché ne réagit pas un-pour-un (décote structurelle du cash vs attentes).
- **Taux 1 an.** $\beta_2 > 0$ sur la période étudiée : co-mouvement taux-marché en *daily* (anomalie par rapport au textbook, mais conforme aux faits sur l'échantillon).

Pourquoi cette métrique R^2 est parlante. Le ratio P/D condense toute l'information de courbe en une seule variable : il agrège l'ensemble des anticipations futures du modèle. Testé contre les vrais mouvements de l'Eurostoxx, il fournit un signal statistique fort ($R^2 \approx 0.603$). Le *cash* ($\Delta \ln D^{(1)}$) ajoute la brique du présent, et Δr_{1Y} capture l'état financier du jour. Ensemble, ces trois moteurs restituent $\sim 60\%$ de la variance quotidienne.

Remarque pratique. La valeur absolue de R^2 dépend de l'horizon, de la fréquence et de l'échantillon. Son interprétation est comparative (entre spécifications) ; ici, l'ajout du ratio P/D issu de la DTS fournit le saut explicatif déterminant.

Test de robustesse par découpage temporel

Pour évaluer la solidité du modèle de structure par terme des dividendes, l'échantillon est scindé en deux périodes : avant-Covid (années avant 2019, calme) et après-Covid (2020 et après, crise). La performance est stable avec $R^2 \approx 65\%$ dans les deux cas.

Période	R^2	β_1 (dividende immédiat)	β_3 (anticipation LT)
Avant-Covid	0.647	1.36	0.42
Covid / Post-Covid	0.644	0.62	1.31

TABLE 3.1 – Robustesse du modèle sur deux périodes

Interprétation.

- **Avant-Covid** : $\beta_1 = 1,35$ indique une sur-réaction au dividende immédiat ; β_3 faible suggère que la composante long terme est moins nécessaire.
- **Covid / Post-Covid** : $\beta_1 = 0,62$ traduit l'effondrement de la confiance dans le dividende immédiat ; $\beta_3 = 1,31$ montre que le marché est piloté par les changements de valorisation et les anticipations futures.

Limites et extensions

Afin de renforcer la robustesse et la crédibilité de notre analyse, il serait pertinent d'étendre la construction de la DTS à un ensemble plus large d'indices, tels que le SPX, le NKY ou encore le HSCEI. Une telle extension permettrait de comparer de manière systématique les dynamiques de *front-loading* par rapport aux structures en *cycle* sur différentes zones géographiques et profils de marché. La confrontation de ces comportements sur plusieurs référentiels offrirait un cadre empirique beaucoup plus riche pour évaluer si les conclusions observées sur le SX5E s'avèrent spécifiques à cet indice ou, au contraire, généralisables. Et donc on pourrait alors affiner les paramètres du modèle en conséquence.

Par ailleurs, intégrer des sources listées, notamment celles provenant d'Eurex, contribuerait à améliorer la qualité et la fiabilité des données utilisées. L'ajout de prix observés sur des marchés organisés, en complément des données OTC, permettrait non seulement de réduire le risque de biais lié au marquage, mais également d'obtenir une vision plus complète de la structure de dividendes implicites.

Enfin, dans l'idéal, l'estimation du profil de pondération temporelle w_t pourrait être menée de manière entièrement empirique. Une approche fondée sur la normalisation quotidienne des dividendes par année d'expiration, plutôt que sur l'utilisation d'un profil *ad hoc*, offrirait une calibration plus réaliste et plus fidèle à la dynamique réelle des dividendes implicites. Cela permettrait notamment de capturer les déformations intramaturités et les éventuelles asymétries temporelles qui ne peuvent être reproduites avec un schéma prédéfini.

Conclusion

Ce mémoire est parti d'une question de contrôle : objectiver la structure par terme des dividendes afin de disposer d'une référence indépendante face aux marquages du *Front Office*. Le parcours suivi est allé de la mesure empirique à la modélisation structurelle, en passant par deux tentatives de monétisation qui n'ont pas abouti mais qui ont orienté la suite.

Le premier chapitre a établi une lecture par maturité de la sensibilité des dividendes au *Spot*. Sur l'Euro Stoxx 50, la corrélation croît nettement avec l'échéance et le bêta converge vers l'unité dès deux à trois ans, signe qu'une prime de risque commune domine progressivement l'aléa spécifique de versement ; sur le SPX, le HSCEI et le NKY, les bêtas restent inférieurs à un aux maturités courtes et moyennes. Corroborée par des données de marché indépendantes, cette *beta curve* est directement utilisable pour challenger un marquage. Le volet volatilité s'est révélé plus fragile : la relation entre volatilité de dividendes et volatilité *Spot* n'est lisible qu'à partir de trois à cinq ans, et la calibration SABR stabilise l'estimation sans renverser ce diagnostic.

Le deuxième chapitre a montré les limites de cette lecture dès qu'il s'agit de prendre position. Malgré une neutralisation au premier ordre à l'entrée, la *mean-reversion* du bêta laisse subsister un terme d'interaction entre la dérive du bêta et le mouvement du spot, qui rend le résultat dépendant du chemin ; la validation en trois périodes confirme l'absence de généralisation hors échantillon. Le *steepener* a échoué pour une raison plus conceptuelle : la pente de bêta est une statistique de rendements sous $\mathbb{P}$, alors que les prix de *DivSwaps* traduisent des anticipations sous $\mathbb{Q}$, et le résultat reste dominé par le *carry* et le *roll* de la courbe.

Le troisième chapitre répond par un modèle estimé en valeurs présentes. Deux facteurs latents à retour à la moyenne, filtrés par Kalman, reconstruisent la forme de la courbe avec une erreur absolue moyenne de l'ordre de 1,93 %, très loin devant le *benchmark* à un facteur. La validation économique vient du *nowcasting* : près de 60 % de la variation quotidienne de l'indice s'explique par le dividende de l'année courante, le taux court et le ratio prix/dividende implicite, ce dernier portant le coefficient le plus élevé. Le découpage pré/post-Covid confirme cette stabilité, tout en révélant une bascule du dividende immédiat vers les anticipations de long terme.

Ces travaux fournissent ainsi deux outils au contrôle du marquage : une courbe de bêta par maturité et une structure par terme reconstruite, lisible en termes de primes d'horizon. Leur extension à d'autres indices, l'intégration de sources listées et une estimation empirique du profil de saisonnalité en constituent les prolongements naturels. Au-delà des résultats, ce stage m'aura appris à mesurer l'écart entre une relation statistique significative et une prime réellement monétisable.

Bibliographie

- A Complete Guide to Using the Nelson–Siegel Model* (2026). Investopedia. URL : <https://www.investopedia.com/terms/n/nelson-siegel-model.asp> (visité le 09/01/2026).
- Article 105 — CRR* (2013). Règlement (UE) n° 575/2013. URL : https://www.doctrine.fr/d/REGLEMENT_EU/2013/575/Article_105 (visité le 09/01/2026).
- BENNETT, Colin et al. (11 oct. 2010). *Dividend Swaps and Dividend Futures*. Rapp. tech. A guide to index and single stock dividend trading. London : Barclays Capital. URL : <https://www.trading-volatility.com/Dividend%20Swaps%20and%20Futures%20-%20Colin%20Bennett.pdf> (visité le 09/01/2026).
- DIEBOLD, Francis X. et Canlin LI (2006). “Forecasting the Term Structure of Government Bond Yields”. In : *International Journal of Forecasting* 22.3, p. 497-517. ISSN : 0169-2070. URL : <https://www.sciencedirect.com/science/article/pii/S0169207006000129> (visité le 09/01/2026).
- Dividend Swaps : Dividend Dynamics : Swapping into Equity Derivatives* (2026). Faster-Capital. URL : <https://fastercapital.com/topics/dividend-swaps.html> (visité le 09/01/2026).
- Dividend Term Structure with Kalman Filtering (2 factors)* (2016). American Economic Association (AEA). URL : <https://www.aeaweb.org/conference/2016/retrieve.php?pdfid=20730&tk=9TnHYz8Y> (visité le 09/01/2026).
- INTERNATIONAL ACCOUNTING STANDARDS BOARD (2013). *IFRS 13 : Fair Value Measurement*. Norme IFRS. URL : <https://www.ifrs.org/issued-standards/list-of-standards/ifrs-13-fair-value-measurement/> (visité le 09/01/2026).
- NELSON, Charles R. et Andrew F. SIEGEL (1987). “Parsimonious Modeling of Yield Curves”. In : *Journal of Business* 60.4, p. 473-489. ISSN : 0021-9398.
- nelson_siegel_svensson* (2026). Version latest. Python package (PyPI). URL : <https://pypi.org/project/nelson-siegel-svensson/> (visité le 09/01/2026).
- REBONATO, Riccardo (2004). *Managing Smile Risk*. Chichester : John Wiley & Sons. ISBN : 978-0-470-85813-9.
- SABR volatility model* (2026). Wikipedia. URL : https://en.wikipedia.org/wiki/SABR_volatility_model (visité le 09/01/2026).
- TR8DR (2016). *Information in Volatility Structure [1]*. Blog : Musings on Algorithms, Models, and the Markets. URL : <https://tr8dr.io/> (visité le 09/01/2026).

Annexe

Code SABR (Python)

```
import pandas as pd
import numpy as np
from scipy.optimize import minimize
import matplotlib.pyplot as plt

def extract_future_and_smile(csv_path: str, maturite: int, strikes: list[int]):
    """
    Extrait le forward (future) et la liste des volatilités implicites (smile)
    à partir d'un fichier CSV.

    Paramètres :
    -----
    csv_path : str
        Chemin du fichier CSV.
    maturite : int
        Valeur de la colonne 'Term' à rechercher.
    strikes : list[int]
        Liste des strikes (StrikePercent) pour lesquels on veut récupérer la
        volatilité.

    Retour :
    -----
    tuple :
        (future, smile) où :
        - future : float, valeur de la colonne 'FutureLevel' pour la maturité
        donnée.
        - smile : list[float], liste des valeurs de 'ImpliedVolatility' pour
        chaque strike.
    """
```

```

# Charger le fichier CSV
df = pd.read_csv(csv_path)

# Filtrer les lignes correspondant à la maturité
df_maturity = df[(df['Term'] == maturite) & (df['Underlying'] == "SX5E")]

if df_maturity.empty:
    raise ValueError(f"Aucune ligne trouvée pour la maturité {maturite}")

# Récupérer le forward (première ligne avec la maturité)
future = df_maturity.iloc[0]['ReferenceLevel']

print("Filtrage maturité :", maturite)
print("Nombre de lignes trouvées :", len(df_maturity))
print(df_maturity.head(3))

# Construire la liste des volatilités (smile)
smile = []
for strike in strikes:
    row = df_maturity[(df_maturity['StrikePercent'] == strike) &
(df_maturity['Underlying'] == "SX5E")]
    if row.empty:
        smile.append(None) # ou np.nan
    else:
        smile.append(row.iloc[0]['ImpliedVolatility'])

return future, smile

# -----
# 1. Formule SABR Normal (Hagan expansion)
# -----
def sabr_normal_vol(F, K, T, alpha, rho, nu, beta=0.0):
    if F == K:
        # ATM formula simplifiée
        term1 = alpha * (F ** beta)

```

```

        term2 = (1 + ((2 - 3 * rho**2) * nu**2 * T) / 24)
        return term1 * term2
    else:
        z = (nu / alpha) * (F - K)
        x_z = np.log((np.sqrt(1 - 2 * rho * z + z**2) + z - rho) / (1 - rho))
        denom = (F - K) * (1 + ((2 - 3 * rho**2) * nu**2 * T) / 24)
        return alpha * (F ** beta) * (z / x_z) * denom

# -----
# 2. Fonction objectif pour calibration
# -----
def objective_normal(params, F, T, strikes, market_vols):
    alpha, rho, nu = params
    model_vols = [sabr_normal_vol(F, K, T, alpha, rho, nu) for K in strikes]
    return np.sqrt(np.mean((np.array(model_vols) - np.array(market_vols))**2))

# -----
# 3. Calibration
# -----
def calibrate_sabr_normal(F, T, strikes, market_vols, initial_guess=(0.02, 0.0,
0.5)):
    bounds = [(1e-6, 1.0), (-0.999, 0.999), (1e-6, 5.0)] # alpha, rho, nu
    result = minimize(objective_normal, initial_guess, args=(F, T, strikes,
market_vols),
                      bounds=bounds, method='L-BFGS-B')
    alpha, rho, nu = result.x
    return alpha, rho, nu

# -----
# 1. Formule SABR Lognormale (Hagan 2002)
# -----
def sabr_lognormal_vol(F, K, T, alpha, rho, nu, beta=1.0, b=0.0):
    """
    Volatilité Black (lognormale) selon l'expansion de Hagan et al. (2002).
    - beta in [0,1]; pour lognormal standard, on prend beta=1.0
    - b: déplacement éventuel (pour gérer F ou K très petits/négatifs). Par
    défaut 0.
    Référence: Hagan et al. (2002), "Managing smile risk".
    """

```

```

# Displaced variables si besoin
Fb = F + b
Kb = K + b

# Gestion ATM (Kb = Fb) : forme en temps avec correction d'ordre 1
if np.isclose(Fb, Kb, rtol=1e-12, atol=1e-15):
    # sigma_B(ATM) = alpha * (Fb)^(beta-1) * [1 + ((2 - 3*rho^2)/24)*nu^2*T]
    # NB: pour beta=1, (Fb)^(beta-1) = (Fb)^0 = 1
    base = alpha * (Fb ** (beta - 1.0))
    corrT = 1.0 + ((2.0 - 3.0 * rho**2) * nu**2 * T) / 24.0 # + (rho * beta *
nu * alpha * T / 4.0)
    return base * corrT

# Off-ATM
# Définition z (cf. Hagan 2002):
# z = (nu/alpha) * (Fb - Kb) * ( (Fb*Kb)^((1 - beta)/2) ) / ( (Fb^(1 - beta) -
Kb^(1 - beta)) )
# Lorsque beta -> 1, on passe à la limite (log(Fb/Kb))
one_minus_beta = 1.0 - beta

if np.isclose(beta, 1.0, rtol=1e-12, atol=1e-15):
    # cas beta = 1 (lognormal pur)
    z = (nu / alpha) * np.log(Fb / Kb)
    # x(z) = log( (sqrt(1 - 2 rho z + z^2) + z - rho) / (1 - rho) )
    x_z = np.log((np.sqrt(1.0 - 2.0 * rho * z + z**2) + z - rho) / (1.0 - rho))
    # Correction en temps
    corrT = 1.0 + ((2.0 - 3.0 * rho**2) * nu**2 * T) / 24.0
    # Préfacteur: alpha / ((Fb*Kb)^((1 - beta)/2)) avec beta=1 -> alpha
    return (alpha * (z / x_z)) * corrT
else:
    # beta in (0,1)
    FK_beta = (Fb * Kb) ** (0.5 * one_minus_beta)
    denom = (Fb ** one_minus_beta - Kb ** one_minus_beta)
    z = (nu / alpha) * (Fb - Kb) * (FK_beta / denom)

    # x(z)
    x_z = np.log((np.sqrt(1.0 - 2.0 * rho * z + z**2) + z - rho) / (1.0 - rho))

    # Préfacteur (Hagan 2002) : alpha / ( (Fb*Kb)^((1 - beta)/2) )
    pref = alpha / FK_beta

```

```

# Correction en temps (ordre-1 en T)
corrT = 1.0 + ((2.0 - 3.0 * rho**2) * nu**2 * T) / 24.0

return pref * (z / x_z) * corrT

# -----
# 2. Fonction objectif pour calibration (vols Black)
# -----
def objective_lognormal(params, F, T, strikes, market_vols, beta=1.0, b=0.0):
    alpha, rho, nu = params
    model_vols = [sabr_lognormal_vol(F, K, T, alpha, rho, nu, beta=beta, b=b) for
K in strikes]
    model_vols = np.array(model_vols)
    market_vols = np.array(market_vols)
    return np.sqrt(np.mean((model_vols - market_vols) ** 2))

# -----
# 3. Calibration (vols Black)
# -----
def calibrate_sabr_lognormal(F, T, strikes, market_vols, beta=1.0, b=0.0,
                             initial_guess=(0.20, 0.0, 0.5)):
    """
    Calibre alpha, rho, nu pour la convention lognormale (Black).
    - initial_guess: alpha ~ niveau ATM Black, rho ~ 0, nu ~ 0.5 (à adapter).
    - bounds: alpha>0, rho in [-0.999, 0.999], nu>0.
    """
    bounds = [(1e-8, 5.0), (-0.999, 0.999), (1e-8, 5.0)]
    result = minimize(
        objective_lognormal, initial_guess,
        args=(F, T, strikes, market_vols, beta, b),
        bounds=bounds, method='L-BFGS-B'
    )
    alpha, rho, nu = result.x
    return alpha, rho, nu

# Récupère future et smile du fichier
csv_path = r"C:\Users\\OneDrive - GROUP DIGITAL
WORKPLACE\Documents\Stage\Index_results_1M\SX5E.csv"
maturite = 60

```

```

nom_strikes = [60, 65, 70, 75, 80, 85, 90, 95, 100, 100, 105, 110, 115, 120, 125,
               130, 135, 140, 145, 150]

future, smile = extract_future_and_smile(csv_path, maturite, nom_strikes)
print(f"FUT DIV = {future}")
print(f"SMILE VOL = {smile}")
print(f"length of smile = {len(smile)}")

# Données marché
F = future          # forward divswap
T = maturite/12      # maturité en années
strikes = np.array(nom_strikes)
strikes_abs = strikes*F/100
market_vols = np.array(smile) # vols normales

print(f"F = {F}")
print(f"T = {T}")
print(f"strikes = {strikes}")
print(f"strikes_abs = {strikes_abs}")
print(f"market_vols = {market_vols}")

# Calibration
alpha, rho, nu = calibrate_sabr_lognormal(F, T, strikes_abs, market_vols)
print(f"Paramètres calibrés: alpha={alpha:.6f}, rho={rho:.6f}, nu={nu:.6f}")

# Volatilité ATM calibrée
atm_vol = sabr_lognormal_vol(F, F, T, alpha, rho, nu)
print(f"Volatilité implicite ATM (divswap): {atm_vol:.6f}")

# Calcul des vols du modèle pour chaque strike
model_vols = [sabr_lognormal_vol(F, K_abs, T, alpha, rho, nu) for K_abs in
               strikes_abs]

rmse = np.sqrt(np.mean((np.array(model_vols) - np.array(market_vols))**2))
print(f"RMSE calibration: {rmse:.6f}")

```

```

# Tracé comparatif
plt.figure(figsize=(10, 6))
plt.plot(strikes, market_vols, 'o-', label='Market Smile', color='blue')
plt.plot(strikes, model_vols, 'o-', label='Calibrated SABR smile', color='red')
plt.title(f'Market Smile vs Calibrated SABR smile\nRMSE = {rmse:.6f}')
plt.xlabel('Strike (%)')
plt.ylabel('Implied Volatility (Black)')
plt.legend()
plt.grid(True)
plt.show()

# -----
# Construction de la surface de volatilité
# -----

# Liste des maturités (en mois) à inclure dans la surface
maturities = [ 12, 24, 36, 48, 60] # adapte selon ton fichier CSV

# Structure pour stocker les vols calibrés
surface_data = []

for m in maturities:
    try:
        # Extraction des données pour la maturité m
        future, smile = extract_future_and_smile(csv_path, m, nom_strikes)
        T = m / 12.0 # maturité en années
        strikes_abs = np.array(nom_strikes) * future / 100
        market_vols = np.array(smile)

        # Calibration SABR
        alpha, rho, nu = calibrate_sabr_lognormal(future, T, strikes_abs,
        market_vols)

        # Calcul des vols du modèle pour chaque strike
        model_vols = [sabr_lognormal_vol(future, K_abs, T, alpha, rho, nu) for
        K_abs in strikes_abs]

        surface_data.append(model_vols)

    print(f"Maturité {m}M : alpha={alpha:.4f}, rho={rho:.4f}, nu={nu:.4f}")

```

```

except ValueError as e:
    print(f" Maturité {m}M ignorée : {e}")

# Conversion en array pour la surface
Z = np.array(surface_data) # shape = (len(maturities), len(nom_strikes))
X, Y = np.meshgrid(nom_strikes, np.array(maturities)/12.0)

# -----
# Tracé 3D de la surface
# -----
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
surf = ax.plot_surface(X, Y, Z, cmap='viridis', edgecolor='none')

ax.set_title('SABR Volatility Surface (Calibrated)')
ax.set_xlabel('Strike (%)')
ax.set_ylabel('Maturity (Years)')
ax.set_zlabel('Implied Volatility')
fig.colorbar(surf, shrink=0.5, aspect=10)

plt.show()

```

Code Etape 3 du modèle DTS (Kalman)

```
# -*- coding: utf-8 -*-
"""
Étape 3 – Estimation Kalman
- Modèle 2-facteurs (court/moyen terme)
- Benchmark 1-facteur
Entrée : step2_measures_long.csv (DATE, n, y, PV_n, PV_1, tau)
Sorties :
- step3_states_2f.csv : états lissés x_t, x_tilde_t
- step3_states_1f.csv : état lissé x_t (benchmark)
- step3_diagnostics.csv : LL/obs, MAE par maturité, demi-vies
"""

import numpy as np
import pandas as pd
from typing import List, Dict, Tuple

# -----
# Fichiers (adapter si besoin)
# -----
MEAS_LONG_CSV = r"C:\Users\OneDrive - GROUP DIGITAL\WORKPLACE\Documents\Stage\modele_div\step2_measures_long.csv"
STATES_2F_CSV = r"C:\Users\OneDrive - GROUP DIGITAL\WORKPLACE\Documents\Stage\modele_div\step3_states_2f.csv"
STATES_1F_CSV = r"C:\Users\OneDrive - GROUP DIGITAL\WORKPLACE\Documents\Stage\modele_div\step3_states_1f.csv"
DIAG_CSV = r"C:\Users\OneDrive - GROUP DIGITAL\WORKPLACE\Documents\Stage\modele_div\step3_diagnostics.csv"

START_DATE = ""
END_DATE = ""

# -----
# Lecture des mesures et construction y_t (vecteurs/jour)
# -----
def load_measures_long(path: str) -> List[Dict]:
    """
    Retourne une liste par DATE:
    {'DATE': ts, 'n_list': [2..10 observés], 'y_vec': np.array([...])}
    """
```

```

"""
df = pd.read_csv(path, parse_dates=['DATE'])
# Filtre période
df = df[(df['DATE'] >= START_DATE) & (df['DATE'] <= END_DATE)].copy()
df = df.sort_values(['DATE', 'n'])
out = []
for d, g in df.groupby('DATE'):
    n_list = g['n'].astype(int).tolist()
    y_vec = g['y'].astype(float).values
    # On ne garde que les dates avec au moins 2 mesures
    m = np.isfinite(y_vec)
    if m.sum() >= 2:
        out.append({'DATE': d, 'n_list': np.array(n_list)[m].tolist(),
                    'y_vec': y_vec[m]})

return out

# -----
# Charges H(n) (2-facteurs)
# -----
def loadings_2f(n_list: List[int], L1: float = 2.5) -> np.ndarray:
    """
    Matrice H (len(n_list) x 2) : b1(n), b2(n) = 1 - b1
    b1(n) décroît avec n (facteur court), L1 ~ "longueur" en années.
    """
    n = np.array(n_list, dtype=float)
    b1 = np.exp(- n / L1)          # poids court terme
    b2 = 1.0 - b1                 # poids moyen terme
    H = np.column_stack([b1, b2])
    return H

# -----
# Charges H(n) (1-facteur)
# -----
def loadings_1f(n_list: List[int], L1: float = 4.0) -> np.ndarray:
    """
    Matrice H (len(n_list) x 1) : c(n) lisse, plus grande sur court terme.
    """
    n = np.array(n_list, dtype=float)
    c = np.exp(- n / L1)
    H = c.reshape(-1,1)
    return H

```

```

# -----
# Conversion demi-vie -> phi (AR(1) daily)
# -----
def phi_from_halflife_days(hl_days: float) -> float:
    """
    Pour un AR(1) daily  $x_t = \phi x_{t-1} + \dots$ 
    On approxime  $hl\_days = \ln(2)/\kappa$  avec  $\phi \sim \exp(-1/hl\_days)$ 
    (approximation simple pour du starter)
    """
    hl_days = max(hl_days, 1.0)
    return float(np.exp(-1.0/hl_days))

# -----
# Kalman Filter (2-facteurs) base
# -----
def kalman_2f_run(meas: List[Dict],
                  phi1: float, phi2: float,
                  sigma1: float, sigma2: float,
                  sigma_eps: float, xbar: float,
                  L1_load: float = 2.5) -> Tuple[float, List[np.ndarray],
List[np.ndarray], float]:
    """
    Exécute Kalman (filtre+lissage) pour 2 facteurs avec:
    - Transition:
        
$$x_t = \phi_1 x_{t-1} + (1-\phi_1) x_{\tilde{t}-1} + w_{1,t}$$

        
$$x_{\tilde{t}} = \phi_2 x_{\tilde{t}-1} + (1-\phi_2) \bar{x} + w_{2,t}$$

        
$$Q = \text{diag}(\sigma_1^2, \sigma_2^2)$$

    - Mesure (pour chaque t avec n_list):
        
$$y_t = H(n\_list) @ [x_t, x_{\tilde{t}}] + v_t, \quad v_t \sim N(0, \sigma_{\text{eps}}^2 I)$$

        
$$H(n) = [b_1(n), 1-b_1(n)], \quad b_1(n) = \exp(-n/L1\_load).$$

    Retour:
        (loglik total, filt_states, smooth_states, ll_per_obs)
    """
    # Etat dimension
    dim_x = 2
    # Matrice transition F et vecteur c_t (constante)
    F = np.array([[phi1, (1.0 - phi1)],
                  [0.0, phi2]], dtype=float)
    c = np.array([0.0, (1.0 - phi2)*xbar], dtype=float) # constant drift on
x_tilde

```

```

Q = np.diag([sigma1**2, sigma2**2])

# Initial state
x_pred = np.zeros(dim_x)          # mean
P_pred = np.diag([0.5, 0.5])      # large incertitude initiale

# Stockages
xs_filt = []    # états filtrés
Ps_filt = []    # covariances filtrées
x_preds = []
P_preds = []
loglik_total = 0.0
n_obs_total = 0

# Filtre (forward)
for t in range(len(meas)):
    # 1) Prediction
    x_pred = F @ x_pred + c
    P_pred = F @ P_pred @ F.T + Q

    # 2) Mise à jour mesure
    y_t = meas[t]['y_vec']
    nlist = meas[t]['n_list']
    H_t = loadings_2f(nlist, L1=L1_load)    # (m x 2)
    R_t = (sigma_eps**2) * np.eye(len(nlist))

    # Innovation
    innov = y_t - H_t @ x_pred
    S = H_t @ P_pred @ H_t.T + R_t

    # Gain
    K = P_pred @ H_t.T @ np.linalg.pinv(S)

    # Mise à jour
    x_f = x_pred + K @ innov
    P_f = (np.eye(dim_x) - K @ H_t) @ P_pred

    # Log-vraisemblance incrémentale (Gaussian)
    try:
        sign, logdet = np.linalg.slogdet(S)
        if sign <= 0:
            ll_inc = -1e6

```

```

        else:
            ll_inc = -0.5 * ( innov.T @ np.linalg.solve(S, innov)
                             + logdet + len(nlist)*np.log(2*np.pi) )

    except Exception:
        ll_inc = -1e6

    loglik_total += float(ll_inc)
    n_obs_total += len(nlist)

    # Stockage
    xs_filt.append(x_f.copy())
    Ps_filt.append(P_f.copy())
    x_preds.append(x_pred.copy())
    P_preds.append(P_pred.copy())

    # Next step
    x_pred, P_pred = x_f, P_f

# Lissage RTS (backward)
xs_smooth = [None]*len(meas)
Ps_smooth = [None]*len(meas)
xs_smooth[-1] = xs_filt[-1].copy()
Ps_smooth[-1] = Ps_filt[-1].copy()
for t in range(len(meas)-2, -1, -1):
    P_f = Ps_filt[t]
    P_p = P_preds[t+1]
    A = P_f @ F.T @ np.linalg.pinv(P_p)
    xs_smooth[t] = xs_filt[t] + A @ (xs_smooth[t+1] - x_preds[t+1])
    Ps_smooth[t] = P_f + A @ (Ps_smooth[t+1] - P_p) @ A.T

ll_per_obs = loglik_total / max(1, n_obs_total)
return loglik_total, xs_smooth, xs_filt, ll_per_obs

# -----
# Kalman Filter (1-facteur) simple
# -----
def kalman_1f_run(meas: List[Dict],
                  phi: float,
                  sigma_x: float,
                  sigma_eps: float,
                  L1_load: float = 4.0) -> Tuple[float, List[np.ndarray],
List[np.ndarray], float]:

```

```

"""
Benchmark 1-facteur:
    x_t = phi*x_{t-1} + w_t, var(w)=sigma_x^2
    y_t = H(n_list) * x_t + v_t, var(v)=sigma_eps^2 I
    H(n) = exp(-n/L1_load)
"""

dim_x = 1
F = np.array([[phi]], dtype=float)
Q = np.array([[sigma_x**2]], dtype=float)

x_pred = np.zeros(dim_x)
P_pred = np.array([[0.5]])

xs_filt, Ps_filt = [], []
x_preds, P_preds = [], []

loglik_total = 0.0
n_obs_total = 0

for t in range(len(meas)):
    # Prediction
    x_pred = F @ x_pred
    P_pred = F @ P_pred @ F.T + Q

    # Mesure
    y_t = meas[t]['y_vec']
    nlist = meas[t]['n_list']
    H_t = loadings_1f(nlist, L1=L1_load) # (m x 1)
    R_t = (sigma_eps**2) * np.eye(len(nlist))

    innov = y_t - H_t @ x_pred
    S = H_t @ P_pred @ H_t.T + R_t
    K = P_pred @ H_t.T @ np.linalg.pinv(S)
    x_f = x_pred + K @ innov
    P_f = (np.eye(dim_x) - K @ H_t) @ P_pred

    # Log-likelihood
    try:
        sign, logdet = np.linalg.slogdet(S)
        if sign <= 0:
            ll_inc = -1e6

```

```

        else:
            ll_inc = -0.5 * ( innov.T @ np.linalg.solve(S, innov)
                             + logdet + len(nlist)*np.log(2*np.pi) )

    except Exception:
        ll_inc = -1e6

    loglik_total += float(ll_inc)
    n_obs_total  += len(nlist)

    xs_filt.append(x_f.copy())
    Ps_filt.append(P_f.copy())
    x_preds.append(x_pred.copy())
    P_preds.append(P_pred.copy())
    x_pred, P_pred = x_f, P_f

# RTS smoothing
xs_smooth = [None]*len(meas)
Ps_smooth = [None]*len(meas)
xs_smooth[-1] = xs_filt[-1].copy()
Ps_smooth[-1] = Ps_filt[-1].copy()
for t in range(len(meas)-2, -1, -1):
    P_f = Ps_filt[t]
    P_p = P_preds[t+1]
    A   = P_f @ F.T @ np.linalg.pinv(P_p)
    xs_smooth[t] = xs_filt[t] + A @ (xs_smooth[t+1] - x_preds[t+1])
    Ps_smooth[t] = P_f + A @ (Ps_smooth[t+1] - P_p) @ A.T

ll_per_obs = loglik_total / max(1, n_obs_total)
return loglik_total, xs_smooth, xs_filt, ll_per_obs

# -----
# Diagnostics : MAE par n, etc.
# -----
def compute_mae_by_maturity(meas: List[Dict],
                            xs_filt: List[np.ndarray],
                            model: str = '2f',
                            L1_load: float = 2.5) -> Dict[int, float]:
    """
    MAE des résidus  $y - H \hat{x}$ , par maturité  $n$ .
    """
    err_by_n: Dict[int, List[float]] = {}

```

```

for t in range(len(meas)):
    y_t = meas[t]['y_vec']
    nlist = meas[t]['n_list']
    xhat = xs_filt[t]
    if model == '2f':
        H_t = loadings_2f(nlist, L1=L1_load)
    else:
        H_t = loadings_1f(nlist, L1=L1_load)
    resid = y_t - H_t @ xhat
    for ni, e in zip(nlist, resid):
        err_by_n.setdefault(int(ni), []).append(float(e))
mae = {n: float(np.nanmean(np.abs(errs))) for n, errs in err_by_n.items()}
return mae

# -----
# Pipeline Étape 3 (run)
# -----
if __name__ == "__main__":
    # 1) Charger les mesures
    meas = load_measures_long(MEAS_LONG_CSV)
    dates = [m['DATE'] for m in meas]

    # 2) Grid search simple (2-facteurs)
    grid_hl_short = [90, 120, 180, 240, 360, 480, 540, 720] # jours ~ 3-12
    mois
    grid_hl_mid = [900, 1050, 1200, 1350, 1500, 1650, 1800, 2000] # jours ~
    2.5-4.1 ans
    grid_sigma1 = [0.02, 0.05, 0.10]
    grid_sigma2 = [0.01, 0.03, 0.06]
    grid_sigmaeps = [0.003, 0.006, 0.010]
    grid_xbar = [-0.06, -0.03, 0.00]

    best_ll, best_params, best_run = -np.inf, None, None
    for hl_s in grid_hl_short:
        for hl_m in grid_hl_mid:
            phi1 = phi_from_half-life_days(hl_s)
            phi2 = phi_from_half-life_days(hl_m)
            for s1 in grid_sigma1:
                for s2 in grid_sigma2:
                    for se in grid_sigmaeps:
                        for xb in grid_xbar:

```

```

        ll, xs_smooth, xs_filt, ll_obs = kalman_2f_run(
            meas, phi1, phi2, s1, s2, se, xb, L1_load=2.5
        )
        if ll > best_ll:
            best_ll = ll
            best_params = {'hl_short_days': hl_s,
                           'hl_mid_days': hl_m,
                           'phi1': phi1, 'phi2': phi2,
                           'sigma1': s1, 'sigma2': s2,
                           'sigma_eps': se, 'xbar': xb,
                           'll_per_obs': ll_obs}
            best_run = (xs_smooth, xs_filt)

xs2_smooth, xs2_filt = best_run
# 3) MAE (2-facteurs)
mae_2f = compute_mae_by_maturity(meas, xs2_filt, model='2f', L1_load=2.5)

# 4) Benchmark 1-facteur (grid simple)
grid_hl_1f = [120, 240, 360, 540]
grid_sx_1f = [0.03, 0.06, 0.10]
grid_se_1f = [0.003, 0.006, 0.010]

best_ll_1f, best_params_1f, best_run_1f = -np.inf, None, None
for hl in grid_hl_1f:
    phi = phi_from_halflife_days(hl)
    for sx in grid_sx_1f:
        for se in grid_se_1f:
            ll1, xs1_smooth, xs1_filt, ll_obs1 = kalman_1f_run(
                meas, phi, sx, se, L1_load=4.0
            )
            if ll1 > best_ll_1f:
                best_ll_1f = ll1
                best_params_1f = {'hl_days': hl, 'phi': phi,
                                   'sigma_x': sx, 'sigma_eps': se,
                                   'll_per_obs': ll_obs1}
                best_run_1f = (xs1_smooth, xs1_filt)

xs1_smooth, xs1_filt = best_run_1f
mae_1f = compute_mae_by_maturity(meas, xs1_filt, model='1f', L1_load=4.0)

# 5) Sauvegarde des états lissés

```

```

df_states_2f = pd.DataFrame({
    'DATE': dates,
    'x_short': [float(x[0]) for x in xs2_smooth],
    'x_medium': [float(x[1]) for x in xs2_smooth]
})
df_states_1f = pd.DataFrame({
    'DATE': dates,
    'x_1f': [float(x[0]) for x in xs1_smooth]
})
df_states_2f.to_csv(STATES_2F_CSV, index=False)
df_states_1f.to_csv(STATES_1F_CSV, index=False)

# 6) Diagnostics comparatifs
# Demi-vies
hl_short = best_params['hl_short_days']
hl_mid = best_params['hl_mid_days']
# MAE moyens (maturités observées)
mae2_mean = float(np.nanmean(list(mae_2f.values()))))
mae1_mean = float(np.nanmean(list(mae_1f.values()))))

diag_rows = []
diag_rows.append({
    'model': '2f',
    'll_per_obs': best_params['ll_per_obs'],
    'hl_short_days': hl_short,
    'hl_mid_days': hl_mid,
    'sigma1': best_params['sigma1'],
    'sigma2': best_params['sigma2'],
    'sigma_eps': best_params['sigma_eps'],
    'xbar': best_params['xbar'],
    'mae_mean': mae2_mean
})
diag_rows.append({
    'model': '1f',
    'll_per_obs': best_params_1f['ll_per_obs'],
    'hl_days': best_params_1f['hl_days'],
    'sigma_x': best_params_1f['sigma_x'],
    'sigma_eps': best_params_1f['sigma_eps'],
    'mae_mean': mae1_mean
})
# MAE par maturité (on ajoute colonnes supplémentaires)

```

```

for n in range(2,11):
    diag_rows[0][f'mae_n{n}'] = mae_2f.get(n, np.nan)
    diag_rows[1][f'mae_n{n}'] = mae_1f.get(n, np.nan)

df_diag = pd.DataFrame(diag_rows)
df_diag.to_csv(DIAG_CSV, index=False)

# 7) Affichages synthèse
print("== Modèle 2-facteurs (best) ==")
print(best_params)
print("MAE (moyen) 2f :", mae2_mean)
print("== Benchmark 1-facteur (best) ==")
print(best_params_1f)
print("MAE (moyen) 1f :", mae1_mean)
print(f"\nÉcrits :\n- {STATES_2F_CSV}\n- {STATES_1F_CSV}\n- {DIAG_CSV}")

```